Paradigmi di Programmazione

Diego Stefanini — prof.ssa Chiara Bodei, a.a. 2026-27

Indice

1 Il lambda calcolo 3
1.1 Calcolabilità e paradigmi 3
1.2 Sintassi dei λ-termini 5
1.3 Variabili libere e legate 7
1.4 Sostituzione e β-riduzione 8
1.5 Ordine di riduzione e strategie 12
1.6 Funzioni con più argomenti 15
1.7 Ricorsione: il combinatore Y 16
1.8 Codifiche: booleani e numeri 17
2 Controllo dei tipi 20
2.1 Perché servono i tipi 20
2.2 Sistemi di tipi 21
2.3 Regole di inferenza 22
2.4 Un linguaggio di espressioni 23
2.5 Type safety 25
3 Il lambda calcolo tipato 26
3.1 Tipi e regole di tipo 26
3.2 Type safety nel λ-calcolo tipato 28
3.3 Estensioni e ricorsione 28
3.4 Cosa possono garantire i tipi 29
4 OCaml: espressioni, funzioni e tipi 31
4.1 Il toplevel e i tipi di base 31
4.2 Funzioni 32
4.3 Condizioni e ricorsione 34
4.4 Type inference e polimorfismo 35
5 Liste e pattern matching 38
5.1 Liste 38
5.2 Pattern matching 39
5.3 Ricorsione sulle liste 42
5.4 Funzioni di ordine superiore sulle liste 44
Riepilogo delle regole 50

1 Il lambda calcolo

Cosa devi saper fare

- mettere le parentesi sottintese e disegnare l'albero di un termine
- trovare le variabili libere e legate, calcolare FV
- fare una sostituzione con l'α-conversione quando serve
- trovare i redex e ridurre fino alla forma normale, o accorgersi che la riduzione non termina
- ridurre con call-by-value e call-by-name e dire in cosa differiscono
- dimostrare che due termini sono β-equivalenti
- usare Y per la ricorsione e calcolare con le codifiche (booleani, IF, numerali di Church)

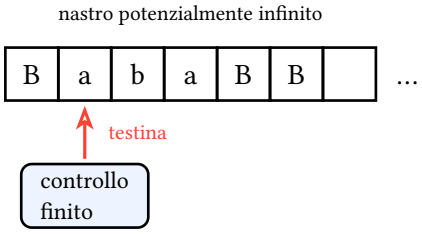
1.1 Calcolabilità e paradigmi

Hilbert, Entscheidungsproblem (problema della decisione): esiste una procedura **del tutto meccanica** che, data una qualunque formula della logica del primo ordine (un'affermazione matematica scritta in un linguaggio formale), dice se è un teorema, cioè se si può dimostrare?

Per rispondere bisogna prima dire cos'è una «procedura meccanica», cioè un **algoritmo**. Tre risposte diverse, quasi insieme:

lambda calcolo	**funzioni ricorsive**	**macchina di Turing**
Alonzo Church, 1935	Kurt Gödel, Stephen Kleene, 1935	Alan Turing, 1936

Com'è fatta la **macchina di Turing**:



- Macchina **ideale**, non fisica: legge e scrive simboli su un nastro, secondo regole fissate.
- Esempio di regola: *legge a, riscrive a e si sposta a destra.*
- È il modello astratto di una macchina che esegue algoritmi.

Macchina di Turing Universale (UTM): una macchina di Turing che, data la **descrizione** di un'altra macchina e i suoi dati, la simula. È il modello teorico del computer: il programma è un dato.

Con la UTM Turing risponde **no** all'Entscheidungsproblem: esistono problemi che nessuna macchina di calcolo può risolvere.

I tre formalismi, anche se sono fatti in modo diversissimo, calcolano esattamente le stesse funzioni. Da qui la **tesi di Church-Turing**. Nella figura, i modelli meno potenti sono quelli che calcolano solo **funzioni totali** (danno sempre un risultato, per ogni input) e, ancora più dentro, le espressioni regolari:



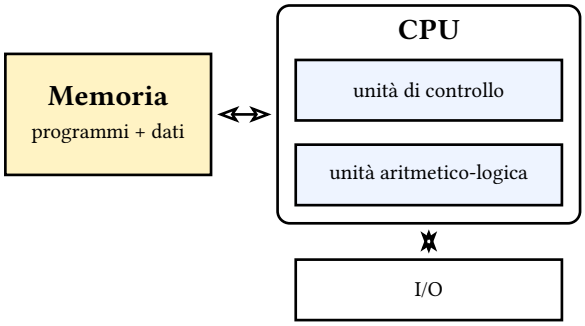
più è interno, meno è potente

Se una funzione è calcolabile con un qualunque formalismo, allora esiste una macchina di Turing che la calcola.

È **indimostrabile** (i formalismi possibili sono infiniti) ma universalmente accettata: non si conosce nessun modello più potente.

Un linguaggio è **Turing completo** se calcola tutto ciò che calcola una macchina di Turing. Si dimostra scrivendo nel linguaggio un programma che **simula** una macchina di Turing.

Dai due modelli, Turing e Church, nascono due famiglie di linguaggi. Da Turing vengono i linguaggi **imperativi**, passando per la macchina di von Neumann:



Architettura di von Neumann, ispirata alla macchina di Turing ma concreta:

- **memoria** con programmi e dati;
- **CPU** che preleva un'istruzione alla volta, la interpreta e la esegue.

La generalità viene dal **programma memorizzato**: il programma sta in memoria come un dato qualsiasi, quindi la stessa macchina esegue qualunque programma basta caricarlo.

Un linguaggio «alla von Neumann» riproduce ad alto livello questa struttura:

Nel linguaggio		Nella macchina
variabili	↔	celle di memoria (il nastro)
istruzioni di controllo (if, cicli)	↔	test & jump (controllo una condizione e salto)
assegnamento	↔	modifica dello stato (leggo e scrivo la memoria)

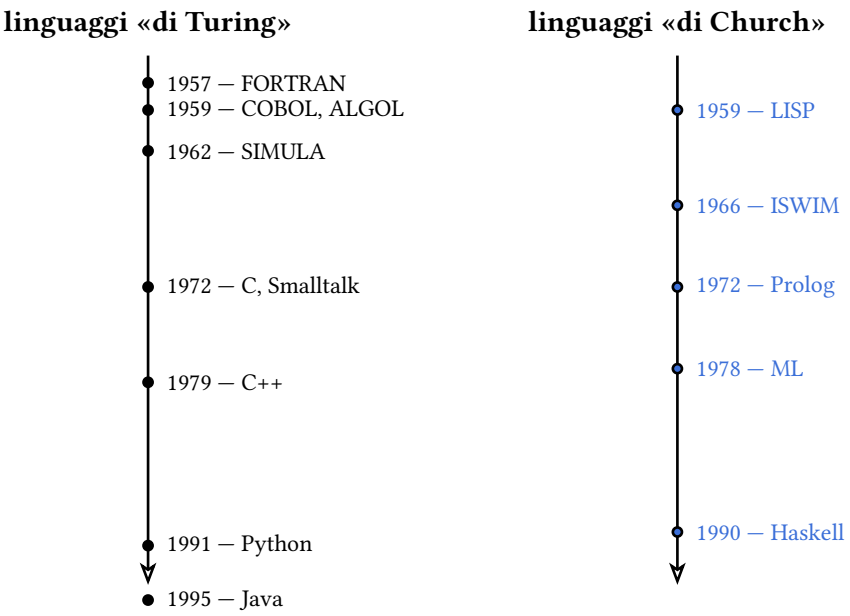
Backus: l'assegnamento divide la programmazione in due mondi.

espressioni
spazio matematico ordinato, con proprietà algebriche utili. Lì avviene la maggior parte del calcolo

istruzioni
spazio disordinato, con poche proprietà utili. La programmazione strutturata prova a metterci un po' d'ordine

Da Church vengono invece i linguaggi **funzionali**.

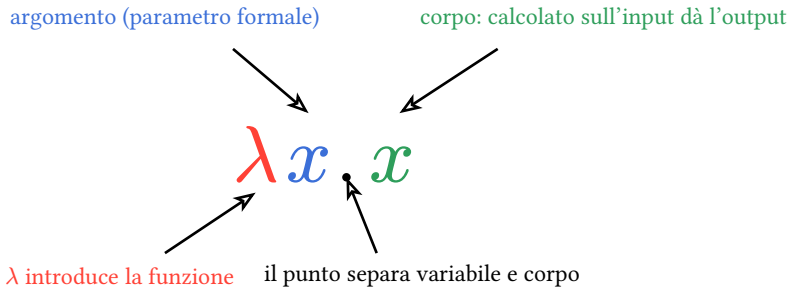
Programmazione funzionale: il programma è una serie di **valutazioni di funzioni matematiche**. Punto di forza: niente **effetti collaterali**. Una funzione ha un effetto collaterale quando, oltre a restituire un valore, cambia qualcosa fuori da sé (una variabile globale, un file, lo schermo). Senza effetti collaterali una funzione con lo stesso input dà sempre lo stesso output, quindi è più facile verificare che il programma sia corretto e ottimizzarlo. Il λ-calcolo (Church, 1935) è il primo linguaggio funzionale.



Le due linee di discendenza

1.2 Sintassi dei λ-termini

Il λ-calcolo ha solo due operazioni. La prima è l'**astrazione**, cioè definire una funzione:



La **legge di corrispondenza** dice cosa fa la funzione su ogni input: qui $\forall x.f(x) = x$, cioè a ogni x associa x stesso. È la **funzione identità**.

La seconda è l'**applicazione**. Applicare una funzione = darle un **parametro attuale** al posto del parametro formale. Immagina la funzione come una scatola:

$$z \;\to\; \boxed{\lambda x.x} \;\to\; z \qquad \textbf{identità}\text{: restituisce l'input così com'è}$$

$$z \;\to\; \boxed{\lambda x.y} \;\to\; y \qquad \textbf{costante}\text{: restituisce sempre } y \;\; (\forall x.f(x) = y)$$

$$z, w \;\to\; \boxed{\lambda x.\lambda y.x} \;\to\; z \qquad \textbf{selezione}\text{: prende due argomenti e restituisce il primo}$$

La selezione passo per passo: $((\lambda x.\lambda y.x)z)w \to (\lambda y.z)w \to z$. Nel primo passo metto z al posto di x nel corpo $\lambda y.x$; nel secondo metto w al posto di y nel corpo z, ma y non c'è, quindi w sparisce e resta z.

Queste due operazioni, più le variabili, sono tutto il linguaggio. Un programma è un'espressione (**λ-espressione**). Ci sono solo tre modi di costruirla:

$$
\begin{array}{lll}
e ::= x & & \text{variabile} \\
\quad | \ \lambda x.e & & \text{astrazione funzionale (dichiarazione di funzione)} \\
\quad | \ e \ e & & \text{applicazione (chiamata di funzione)}
\end{array}
$$

Si legge: un'espressione e è una variabile, **oppure** (|) un'astrazione, **oppure** un'applicazione. La definizione è **ricorsiva**: dentro $\lambda x.e$ e dentro $e \ e$ ci sono altre espressioni, costruite con le stesse tre regole. Per esempio $(\lambda x.x)y$ è un'applicazione di $\lambda x.x$ (astrazione, con corpo la variabile x) alla variabile y.

$\lambda x.e$ è una **funzione anonima**: non ha nome, e x è la dichiarazione del suo parametro. Lo stesso concetto nei linguaggi:

Linguaggio	$x \mapsto x + 1$
JavaScript	`function(a){ return a + 1; }` oppure `a => a+1`
OCaml	`fun x -> x+1`
Java (si chiamano lambda)	`(int x) -> x + 1`

Da qui in poi useremo lettere come e, e_1, e_2, e_3. **Non** sono variabili del λ-calcolo: sono nomi che stanno per **un'espressione qualsiasi**, come in algebra $a + b$ vale per due numeri qualsiasi.

In e_1e_2, e_1 è la funzione che chiamo ed e_2 l'argomento che le passo (in JavaScript `e1(e2)`). La funzione non deve per forza avere un nome: la sua definizione può stare **direttamente dentro** la chiamata.

$$
\underbrace{(\lambda x.(\lambda y.xy))}_{e_1 = \text{ la funzione}} \ \underbrace{(\lambda z.z)}_{e_2 = \text{ l'argomento}}
$$

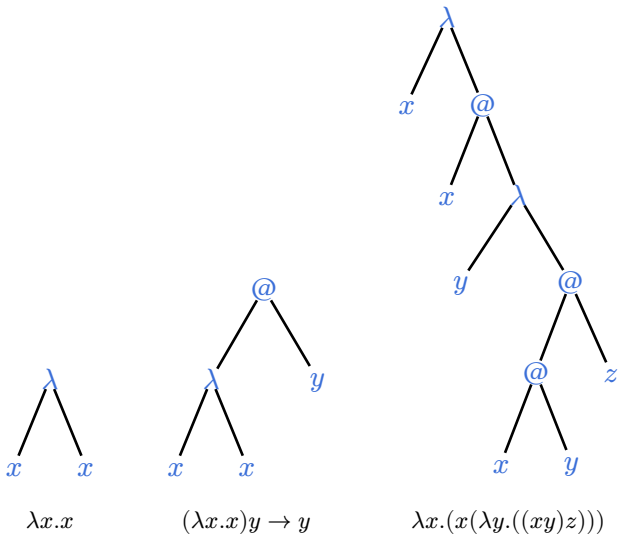
Senza parentesi $\lambda x.xy$ è ambiguo: è $\lambda x.(xy)$ (prende x e restituisce x applicata a y) oppure $(\lambda x.x)y$ (l'identità applicata a y)? Decidono due convenzioni (importanti), che servono anche a scrivere meno parentesi:

1. Lo scope di λ va il più a destra possibile
Il corpo di una λ è **tutto** quello che c'è dopo il punto, fino alla fine (o fino a una parentesi chiusa).
$\lambda x.xy$ è $\lambda x.(xy)$
$\lambda x.\lambda y.xy$ è $\lambda x.(\lambda y.(xy))$

2. L'applicazione associa a sinistra
Con più applicazioni di fila si parte da sinistra: prima e_1 applicata a e_2, poi il risultato applicato a e_3. Come `f(a)(b)` in JavaScript.
$e_1e_2e_3$ è $(e_1e_2)e_3$

Esempio: quali parentesi sono sottintese in $\lambda x.x\lambda y.xyz$?
1. Regola 1. Il corpo di λx è tutto il resto, $x\lambda y.xyz$; dentro, il corpo di λy è xyz: $\lambda x.(x\lambda y.(xyz))$
2. Regola 2. xyz diventa $(xy)z$; e $x\lambda y.\ldots$ è x applicata alla funzione $\lambda y.\ldots$: $\lambda x.(x(\lambda y.((xy)z)))$

Ogni termine si può disegnare come un **albero**, che segue l'annidamento completo delle parentesi. Serve a vedere quali passi di valutazione si possono fare. Nodo λ: figlio sinistro il parametro, destro il corpo. Nodo @ (applicazione): figli funzione e argomento.



1.3 Variabili libere e legate

λ è un **operatore di binding**: in $\lambda x.e$ lega la x dentro e (il suo **scope**), come $\forall x$ in $\forall x.P$ nella logica. È l'**unico** modo di associare valori a variabili nel λ-calcolo.

- **Legata**: introdotta da un λ che la contiene. **verde**, la freccia punta al suo λ.
- **Libera**: nessun λ la dichiara. **rossa**.

Espressione	Chi è legato
$\lambda\,\mathbf{x}\,.\,\mathbf{x}$	x legata
$\lambda\,\mathbf{x}\,.\,\lambda\,\mathbf{y}\,.\,(\,\mathbf{x}\,\mathbf{y}\,\mathbf{z}\,)$	x e y legate, z libera
$(\,\lambda\,\mathbf{f}\,.\,\mathbf{f}\,\mathbf{x}\,)\,\mathbf{y}$	f legata, x e y libere
$(\,\lambda\,\mathbf{f}\,.\,\mathbf{f}\,\mathbf{x}\,)\,\mathbf{f}$	la prima f è legata; x e **la seconda f sono libere**: sta fuori dalle parentesi, fuori dallo scope di λf

Una stessa variabile può voler dire cose diverse in punti diversi:

$$\lambda\,\mathbf{x}\,.\,\mathbf{x}\,(\,\lambda\,\mathbf{x}\,.\,\mathbf{x}\,)\,\mathbf{x}$$

che con le parentesi esplicite è $\lambda x.(x(\lambda x.x)x)$

La x nel λx interno è legata a **quel** λ, non a quello esterno. Tre x, due significati: da qui nasce il conflitto di nomi.

Applicando senza fare attenzione ai nomi nasce un problema:

	Espressione	Risultato
ingenuo	$(\lambda x.\lambda y.xy)y$	$\lambda y.yy$ ✗ sbagliato
rinomino y in z	$(\lambda x.\lambda z.xz)y$	$\lambda z.yz$ ✓

Le due espressioni di partenza sono **la stessa funzione** (ho solo cambiato il nome del parametro), ma i risultati si comportano in modo diverso. Nel primo caso la y libera che passo come argomento finisce **catturata** dal λy interno: è un **conflitto di nomi**.

x in $\lambda x.e$ è un segnaposto: si può rinominare con una variabile **fresca**, cioè un nome che non compare da nessuna parte nelle espressioni che stiamo trattando. È così che si risolve il conflitto di nomi, come nella riga «rinomino» della tabella.

Le variabili libere si possono definire in modo preciso, con una **definizione induttiva**: un caso per ogni elemento della sintassi.

$\mathrm{FV}(x) = \{x\}$	variabile
$\mathrm{FV}(e_1 e_2) = \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2)$	applicazione: le libere di tutte e due
$\mathrm{FV}(\lambda x.e) = \mathrm{FV}(e) \setminus \{x\}$	astrazione: il λ lega x, la tolgo

FV sta per *Free Variables*. Una variabile che non è libera è **legata** (*bound*). Un termine senza variabili libere ($\mathrm{FV}(e) = \varnothing$) si dice **chiuso**; i termini chiusi si chiamano **combinatori**, il più semplice è l'identità $\lambda x.x$.

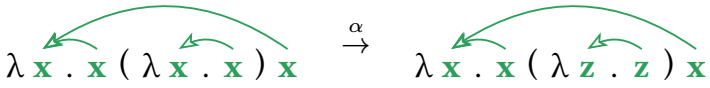
Esempio: $\mathrm{FV}((\lambda x.\lambda y.xy)((\lambda z.z)k))$

$$= \mathrm{FV}(\lambda x.\lambda y.xy) \cup \mathrm{FV}((\lambda z.z)k) = \varnothing \cup \{k\} = \{k\}$$

Cambiare nome a un parametro non cambia niente: $\lambda a.ac$ e $\lambda b.bc$ si dicono **α-equivalenti**, perché a e b contano solo per il **ruolo** che hanno nell'espressione. Lo stesso vale nei linguaggi:

`function(a){ return a + 1; }` è α-equivalente a `function(b){ return b + 1; }`

Espressioni α-equivalenti rappresentano **lo stesso programma**. Rinominare una variabile legata con una variabile fresca si chiama **α-conversione**. Serve a passare da $\lambda x.x$ a $\lambda z.z$, e soprattutto a togliere il conflitto di nomi visto sopra (*variable shadowing*):



i due termini sono equivalenti a meno di α-conversione, ma nel secondo ogni nome ha un solo significato

Il nome nuovo deve essere **fresco**. $\lambda x.xy$ e $\lambda y.yy$ **non** sono α-equivalenti: nel primo la y è libera, nel secondo è stata catturata dal λy ed è diventata il parametro. La prima funzione applica l'argomento a y, la seconda applica l'argomento a se stesso.

Esercizio 1
Trovare le variabili libere (prima mettere le parentesi sottintese):
a. $(\lambda x.x)(\lambda y.xx)(\lambda y.y)y$
b. $(\lambda x.xk)((\lambda x.\lambda y.xy)(\lambda z.(\lambda x.z)y))$
c. $x(\lambda x.(\lambda y.yy)x)y$

1.4 Sostituzione e β-riduzione

Cosa vuol dire **eseguire** (valutare) una λ-espressione? La valutazione, *eval*, fa solo una cosa: **chiamare funzioni**.

$\mathrm{eval}((\lambda x.e_1)e_2)$: rimpiazzo ogni occorrenza di x in e_1 con e_2, poi valuto il termine che ne risulta.

La **sostituzione** si scrive $e_1\{x := e_2\}$ (oppure $e_1\{e_2/x\}$): è e_1 con ogni occorrenza **libera** di x sostituita da e_2. Per esempio $xz\{x := \lambda y.y\} = (\lambda y.y)z$.

Il problema: e se e_2 contiene una variabile libera che in e_1 è legata? Sostituendo alla cieca finisce **catturata** dal λ di e_1, e legare una variabile libera cambia il significato. Esempio con le operazioni aritmetiche, per semplicità: $(\lambda x.(x * y))\{y := (x + x)\}$.

Come	**Risultato**
alla cieca	$\lambda x.(x * (x + x))$ ✗ le x di $x + x$ ora sono il parametro
α-conversione, z fresca	$(\lambda z.(z * y))\{y := (x + x)\} = \lambda z.(z * (x + x))$ ✓

La soluzione è la **sostituzione che evita la cattura** (*capture-avoiding substitution*): prima di sostituire, se serve, rinomino con l'α-conversione. La definizione ha un caso per ogni elemento della sintassi:

	Regola	**Perché**
variabile	$x\{x := e\} \equiv e$	è proprio la variabile da sostituire
	$y\{x := e\} \equiv y$ se $x \neq y$	un'altra variabile non si tocca
applicazione	$(e_1 e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$	sostituisco da tutte e due le parti
astrazione	$(\lambda x.e_1)\{x := e\} \equiv \lambda x.e_1$	qui x è legata: la sostituzione vale solo per le variabili libere, quindi non cambia niente
	$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\})$ se $y \neq x$ e $y \notin \mathrm{FV}(e)$	in e non compare y: nessun rischio di conflitto
	$(\lambda y.e_1)\{x := e\} \equiv \lambda z.((e_1\{y := z\})\{x := e\})$ se $y \neq x$ e $y \in \mathrm{FV}(e)$, z fresca	la y libera di e verrebbe catturata: prima α-converto y in z, poi sostituisco

«Fresca» qui vuol dire che non compare né in e_1 né in e. L'ultimo caso, disegnato: sostituire x con y nel corpo di $\lambda x.\lambda y.xy$.



alla cieca: la y arriva in un mondo dove esiste già una y legata e diventa indistinguibile da lei ✗



prima rinomino y in z: ora la y resta libera ✓

Un esempio completo: $(\lambda x.\lambda y.((\lambda z.z)x))y$. Il passo $(\lambda y.((\lambda z.z)x))\{x := y\}$ fatto alla cieca **non va bene**: le due y sono diverse ($y \in \mathrm{FV}(e)$). Quindi, con k fresca:

$$(\lambda y.((\lambda z.z)x))\{x := y\} \equiv_\alpha (\lambda k.((\lambda z.z)x))\{x := y\} = \lambda k.((\lambda z.z)y)$$

Esempi. L'espressione è sempre la stessa, cambia la variabile da sostituire ($\equiv_\alpha$ vuol dire «α-equivalente», cioè uguale a meno di rinominare):

Sostituzione	**Risultato**
$((\lambda x.yx)w)\{x := \lambda k.kx\}$	$(\lambda x.yx)w$ l'unica x è legata: $(\lambda x.e_1)\{x := e\} \equiv \lambda x.e_1$
$((\lambda x.yx)w)\{y := \lambda k.kx\}$	$\equiv_\alpha ((\lambda z.yz)w)\{y := \lambda k.kx\} = (\lambda z.(\lambda k.kx)z)w$ la x di $\lambda k.kx$ è libera e verrebbe catturata da λx: rinomino
$((\lambda x.yx)w)\{w := \lambda k.kx\}$	$(\lambda x.yx)(\lambda k.kx)$ w non è sotto nessun λ: caso applicazione

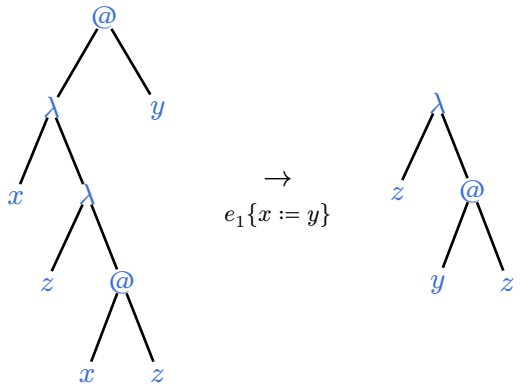
Esercizio 2
Calcolare le sostituzioni (che evitano la cattura):
a. $(\lambda y.xy)\{x := y\}$
b. $(\lambda y.xy)\{x := \lambda y.y\}$
c. $(\lambda y.xy)\{x := \lambda x.xy\}$

Con la sostituzione si scrive la regola fondamentale del λ-calcolo, la **β-riduzione**:

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\}$$

- Cattura esattamente l'**applicazione di funzione**: $(\lambda x.x)3 \to 3$.
- Un **redex** (espressione riducibile) è una sottoespressione della forma $(\lambda x.e_1)e_2$, a cui la regola si può applicare.

Esempio: $(\lambda x.\lambda z.xz)y$. Il parametro formale è x, il corpo è $e_1 = \lambda z.xz$, il parametro attuale è $e_2 = y$:



$$(\lambda x.\lambda z.xz)y \to \lambda z.yz$$

Il λ-calcolo è di **ordine superiore**: una funzione può prendere funzioni come parametri e restituire funzioni come risultato, in modo naturale.

Espressione	**Cosa fa**
$\lambda x.x$	identità
$\lambda y.(\lambda x.x)$	scarta l'argomento y e restituisce l'identità: è una **funzione costante**
$\lambda f.f(\lambda x.x)$	data una funzione f, la **invoca sull'identità**

Applicazione	Passi
$(\lambda x.x)y$	$\to y$
$(\lambda x.x)(\lambda y.y)$	$\to \lambda y.y$ l'identità applicata all'identità
$(\lambda x.xy)z$	$\to zy$ prende una funzione z e la applica a y
$(\lambda x.xy)(\lambda z.z)$	$\to (\lambda z.z)y \to y$ il parametro attuale è l'identità
$(\lambda x.((\lambda y.y)x))z$	$\to (\lambda y.y)z \to z$
$(\lambda f.fz)(\lambda x.x)$	$\to (\lambda x.x)z \to z$ prende una funzione e la applica a z
$(\lambda x.(xx))(\lambda y.y)$	$\to (\lambda y.y)(\lambda y.y) \to \lambda y.y$ due termini uguali con ruoli diversi: funzione e argomento
$((\lambda x.\lambda y.x+y)3)5$	$\to (\lambda y.3+y)5 \to 3+5 \to 8$ (supponendo di avere il +)
$((\lambda x.\lambda y.xy)(\lambda x.x))z$	$\to (\lambda y.(\lambda x.x)y)z \to (\lambda x.x)z \to z$ forma $e_1e_2e_3$
$(\lambda x.\lambda y.xy)zk$	$\to (\lambda y.zy)k \to zk$

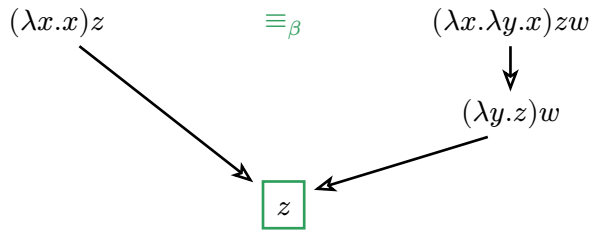
Esercizio 3
Trovare tutti i redex:
a. $(\lambda x.x)((\lambda x.x)(\lambda z.(\lambda x.x)z))$
b. $(\lambda x.x)(\lambda x.xx)(\lambda y.y)$
c. $x((\lambda x.(\lambda y.yy)x)y)$

La valutazione va avanti scegliendo un redex e riducendolo. Quando non ci sono più redex l'espressione è in **forma normale β**: non si può più riscrivere con la β-riduzione, ed è il **risultato finale**, il **valore calcolato**. Per esempio $\lambda x.x$ e $\lambda t.\lambda f.t$ sono valori: **le funzioni sono valori**.

Notazione	Significato
$e_1 \to e_2$	e_2 si ottiene da e_1 con **un** passo di riduzione
$e_1 \Rightarrow e_2$	e_2 si ottiene da e_1 con **zero o più** passi

Un passo di β-riduzione è un passo di calcolo, quindi $\Rightarrow$ (la **chiusura riflessiva e transitiva** di $\to$) rappresenta una computazione qualsiasi. Quando $e_1 \Rightarrow e_2$ si dice che e_1 è **β-riducibile** a e_2.

Con $\Rightarrow$ si definisce quando due espressioni sono «uguali». e_1 ed e_2 sono **β-equivalenti**, $e_1 \equiv_\beta e_2$, se:
1. sono identiche a meno di α-conversione, oppure
2. $e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$, oppure
3. $e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$ (arrivano alla stessa espressione).



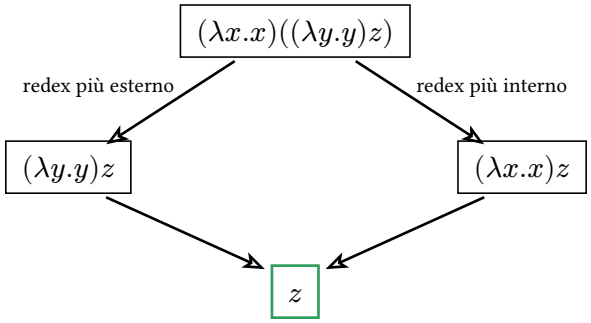
β-equivalenti per il terzo caso: tutte e due si riducono a z

Intuizione: due espressioni sono β-equivalenti quando sono indistinguibili dal punto di vista del calcolo, cioè calcolano gli stessi risultati.

Esercizio 4
Dimostrare le β-equivalenze, con meno passi possibile:
a. $(\lambda x.x)(\lambda y.y)z \equiv_\beta (\lambda x.(\lambda k.k))wz$
b. $(\lambda x.x)(\lambda y.y) \equiv_\beta (\lambda y.y)(\lambda x.x)$
c. $(\lambda x.\lambda y.xy)y \equiv_\beta (\lambda x.(\lambda x.x)(\lambda z.yz))k$

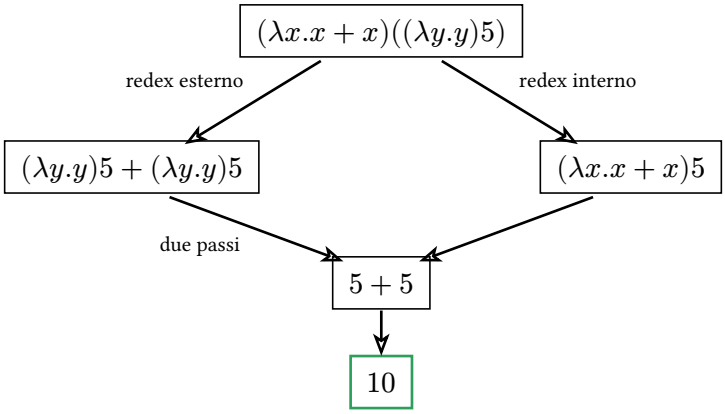
1.5 Ordine di riduzione e strategie

Quando un'espressione contiene più redex, si può scegliere da quale cominciare. In $(\lambda x.x)((\lambda y.y)z)$ ce ne sono due:



Strade diverse, stesso risultato

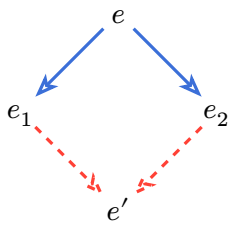
Ma in generale l'ordine di valutazione può cambiare il risultato finale? Un altro esempio (usando il $+$):



Scegliendo prima il redex esterno si copia $(\lambda y.y)5$ e lo si riduce due volte; prima quello interno, una volta sola. Il risultato è lo stesso

La risposta generale è il teorema di Church-Rosser: l'ordine in cui si scelgono le β-riduzioni **non influisce sul risultato finale**.

Teorema di Church-Rosser. Se $e \Rightarrow e_1$ e $e \Rightarrow e_2$, allora esiste e' con $e_1 \Rightarrow e'$ e $e_2 \Rightarrow e'$. Cioè: se da una stessa espressione si fanno due sequenze di riduzioni diverse, c'è sempre un'espressione raggiungibile da entrambi i risultati (anche con zero passi).



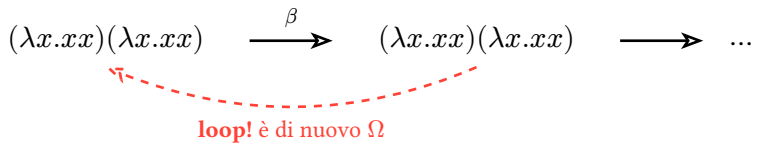
Proprietà di confluenza
(o del diamante)

Per questo nell'esempio sopra entrambe le strade arrivano a 10.

Non sempre però si arriva a una forma normale. Il combinatore

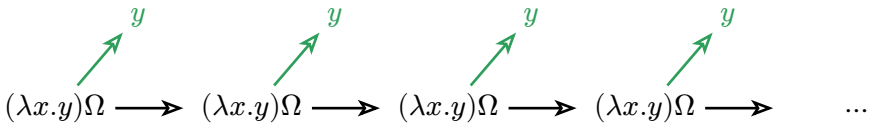
$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

contiene un solo redex, e ridurlo dà di nuovo Ω: la funzione $\lambda x.xx$ applica il suo argomento a se stesso, e l'argomento è proprio $\lambda x.xx$.



Ω non si può ridurre in forma normale: è un combinatore **divergente**, l'equivalente di un ciclo infinito.

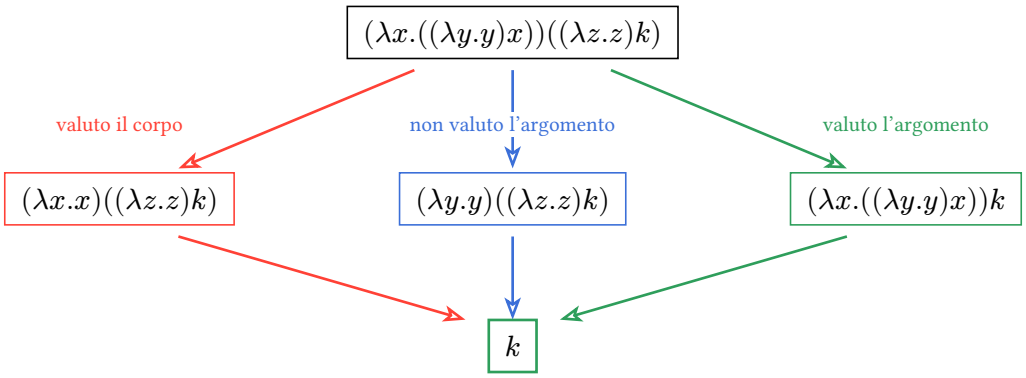
Una stessa espressione può **terminare** facendo certe scelte di riduzione e **non terminare** facendone altre. Per esempio $(\lambda x.y)\Omega$:



In orizzontale riduco dentro Ω e resto sempre fermo; in qualunque momento posso invece applicare $\lambda x.y$, che butta via l'argomento e dà y

Church-Rosser garantisce che **in tutti i casi in cui la riduzione termina, il risultato è lo stesso**: non possono esserci risultati diversi.

Le scelte possibili sono di tre tipi. In $(\lambda x.((\lambda y.y)x))((\lambda z.z)k)$:



Tre scelte diverse, stesso risultato k

Succede lo stesso in un linguaggio. Con `function f(x) { 3+2 + x }` e la chiamata `f(2+1)` posso calcolare l'argomento prima di passarlo (`f(3)`), passarlo non calcolato e lasciarlo alla funzione (`f(2+1)`), oppure semplificare il corpo (`5 + x`).

Una **strategia di valutazione** fissa quale scelta fare. Valutare un'applicazione $(\lambda x.e)e'$ ha tre passi: valuto l'espressione che definisce la funzione, passo il parametro, valuto il corpo. Le due strategie cambiano il secondo passo, il **passaggio dei parametri**:

	Call-by-value (CBV), *eager*	**Call-by-name (CBN), *lazy***
Cosa fa con l'argomento e'	lo valuta **prima** della β-riduzione	fa **subito** la β-riduzione
Al posto di x finisce	il **valore** dell'argomento	l'**espressione non valutata**
Redex scelto	il più interno	il più esterno
Linguaggi	OCaml e la maggior parte	Haskell

Le strategie si scrivono con **regole**: sopra la riga l'ipotesi, sotto la conclusione. La prima a sinistra dice: se e_1 fa un passo e diventa e', allora $e_1 e_2$ diventa $e' e_2$.

Standard (senza strategia)

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\} \quad \text{(App)} \qquad \frac{e \to e'}{\lambda x.e \to \lambda x.e'} \quad \text{(App-Body)}$$

$$\frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \quad \text{(App-1)} \qquad \frac{e_2 \to e'}{e_1 e_2 \to e_1 e'} \quad \text{(App-2)}$$

Call-by-value

$$(\lambda x.e_1)v_2 \to e_1\{x := v_2\} \quad \text{(CV-App)} \qquad \frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \quad \text{(CV-App-1)} \qquad \frac{e_2 \to e'}{v_1 e_2 \to v_1 e'} \quad \text{(CV-App-2)}$$

Call-by-name

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\} \quad \text{(CN-App)} \qquad \frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \quad \text{(CN-App-1)}$$

- **Standard**: qualsiasi redex si può ridurre in qualsiasi momento, anche dentro il corpo di una funzione (App-Body). Non è deterministica.
- **Call-by-value**: v sta per un **valore**, cioè un'espressione che non si può più ridurre: una funzione, ma anche una variabile libera come y o un'applicazione bloccata come xy. La β-riduzione scatta solo quando l'argomento è un valore v_2; l'argomento e_2 si riduce solo quando la funzione è già un valore v_1.
- **Call-by-name**: si riduce la parte funzione finché diventa $\lambda x.e_1$, poi si applica. L'argomento non si tocca.

> CBV e CBN **non hanno la regola che riduce il corpo di una funzione**. Quindi $\lambda x.((\lambda y.y)z)$ con la riduzione standard diventa $\lambda x.z$, con le due strategie resta così com'è: per loro una funzione è già un valore, anche se dentro ha un redex. È quello che succede nei linguaggi: il corpo di una funzione non viene eseguito finché la funzione non viene chiamata.

La stessa espressione con le due strategie (usando numeri e $+$):

Call-by-value: redex più interno

$$\begin{aligned} & (\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1) \\ \to\ & (\lambda x.\lambda y.yx)7(\lambda x.x+1) \\ \to\ & (\lambda y.y7)(\lambda x.x+1) \\ \to\ & (\lambda x.x+1)7 \\ \to\ & 7+1 \to 8 \end{aligned}$$

Call-by-name: redex più esterno

$$\begin{aligned} & (\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1) \\ \to\ & (\lambda y.y(5+2))(\lambda x.x+1) \\ \to\ & (\lambda x.x+1)(5+2) \\ \to\ & (5+2)+1 \\ \to\ & 7+1 \to 8 \end{aligned}$$

Conta quale strategia si sceglie?

- Se la valutazione termina, **no**: per la confluenza il risultato è lo stesso.
- La CBV valuta ogni argomento **una volta sola**. La CBN lo valuta **solo se serve**, ma può valutarlo più volte: è la strada di sinistra nella figura di $(\lambda x.x + x)((\lambda y.y)5)$.
- La CBN **termina ogni volta che è possibile**; la CBV può non terminare anche quando la CBN termina. In $(\lambda x.y)\Omega$ la CBN applica subito $\lambda x.y$ e dà y; la CBV deve prima valutare l'argomento Ω e non finisce mai. Non contraddice la confluenza, che dice che una strada verso il termine comune **esiste**, non quale strategia la trova.

Esiste anche la **call-by-need**, una variante della CBN: l'argomento si passa senza valutarlo, ma la prima volta che serve si valuta e il risultato si memorizza, così le altre occorrenze lo riusano. Termina negli stessi casi della CBN ed evita le valutazioni ripetute (è la valutazione *lazy* di Haskell); non la tratteremo in dettaglio.

Esercizio 5
Ridurre fino alla forma normale, o accorgersi che la riduzione non termina. A ogni passo indicare il redex scelto; se ci sono più scelte, provarle tutte.
a. $(\lambda x.\lambda y.xy)(\lambda z.zy)y$
b. $(\lambda x.\lambda y.xy)(\lambda x.yy)k$
c. $(\lambda x.(\lambda y.xy)(\lambda k.k))(xy)$
d. $(\lambda x.y)((\lambda y.yyy)(\lambda x.xxx))$
e. $(\lambda z.((\lambda x.\lambda y.xy)z)z)(\lambda z.zz)$

Esercizio 6
Ripetere l'esercizio 5 con call-by-value e con call-by-name.

1.6 Funzioni con più argomenti

Nel λ-calcolo ogni funzione prende **un solo argomento**. Una funzione di due argomenti si scrive come **catena** di funzioni di un argomento, come la selezione $\lambda x.\lambda y.x$: la prima prende x e restituisce una funzione che prende y.

$$\lambda x.(\lambda y.xy) \qquad \text{da } \; f : (X \times Y) \to Z \;\; \text{ a } \;\; g : X \to (Y \to Z)$$

Questa trasformazione si chiama **currying** (dal logico Haskell Curry). g prende un X e restituisce una funzione da Y a Z, e calcola gli stessi risultati di f:

$$(\lambda x.\lambda y.x + y)10 \to \lambda y.10 + y \qquad (\lambda y.10 + y)5 = 15$$

$\lambda y.10 + y$ è una funzione a sé: somma 10 a quello che le passi. Dare a una funzione solo il primo argomento si chiama **applicazione parziale**: ottieni una funzione che aspetta gli altri.

L'espressività non cambia, il risultato è lo stesso che avrei con due parametri insieme:

Definizione	**Chiamata**
$F = \lambda(x, y).e$	$F(e_1, e_2) \to e\{x := e_1\}\{y := e_2\}$
$F = \lambda x.\lambda y.e$	$(Fe_1)e_2 \to (\lambda y.e)\{x := e_1\}e_2 \to e\{x := e_1\}\{y := e_2\}$

In JavaScript, `curry` prende una funzione `f` che vuole due argomenti insieme e restituisce una funzione che li prende uno alla volta (nota le funzioni annidate):

```
function curry(f) {
  return function(a) {
    return function(b) {
      return f(a, b);
    };
  };
}
```

```
function sum(a, b) { return a + b; }

let curriedSum = curry(sum);
let h = curriedSum(1); // somma sempre 1
alert( h(2) );         // 3
alert( curriedSum(1)(2) ); // 3
```

`sum` passa da `Int × Int → Int` a `Int → (Int → Int)`. I vantaggi del currying:

- **uniformità**: ogni funzione ha un solo argomento, non serve una regola a parte per più argomenti;
- **applicazione parziale**: funzioni specializzate (come `h`) senza costrutti in più;
- **minimalità**: astrazione e applicazione bastano per funzioni con qualsiasi numero di argomenti.

Tutto questo si regge sull'**ordine superiore** (*higher order*). Altri due esempi: Twice prende una funzione e la applica due volte:

$$\text{Twice} = \lambda f.\lambda x.f(fx)$$

$$\text{Twice}(\lambda y.y+y)2 \Rightarrow (\lambda y.y+y)((\lambda y.y+y)2) \Rightarrow (\lambda y.y+y)4 \Rightarrow 8$$

Comp è la **composizione** $f \circ g$: prende due funzioni e ne restituisce una nuova che applica prima g e poi f. Ha tipo $(B \to C) \times (A \to B) \to (A \to C)$.

$$\text{Comp} = \lambda f.\lambda g.\lambda x.f(gx) \qquad x \to \boxed{g} \to gx \to \boxed{f} \to f(gx)$$

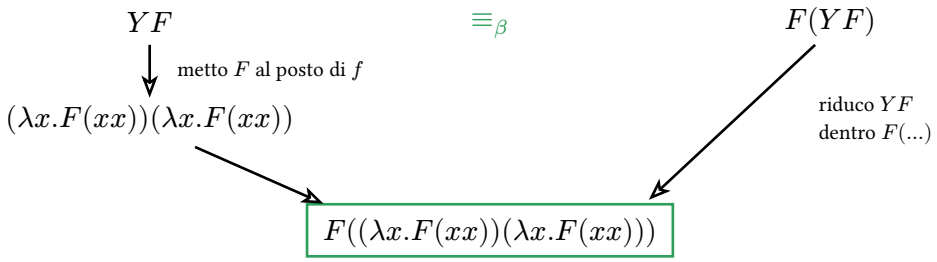
1.7 Ricorsione: il combinatore Y

Nel λ-calcolo non c'è un meccanismo per la ricorsione: le funzioni sono anonime, quindi una funzione non ha un nome con cui richiamare se stessa. Il problema si aggira con il **combinatore di punto fisso** Y.

Un **punto fisso** di una funzione F è un termine p tale che $F(p) = p$. Y è una funzione di ordine superiore che, data F, ne costruisce un punto fisso:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)) \qquad YF \equiv_\beta F(YF)$$

Perché vale la proprietà: YF e $F(YF)$ si riducono allo stesso termine.



Nel secondo passo $\lambda x.F(xx)$ è passata a se stessa come argomento: ogni xx diventa una nuova copia del termine di partenza, con una F davanti. Continuando si ottiene $F(F(\ldots))$: una «copia infinita» della funzione.

Per definire una funzione ricorsiva $F = \langle \text{corpo che contiene } F \rangle$ si fanno due passi:

1. si scrive $G = \lambda f.\langle \text{espressione che contiene } f \rangle$: il nome della funzione diventa un parametro;
2. si definisce $F = YG$.

Allora $F = YG \equiv_\beta G(YG)$, cioè l'espressione con YG al posto di f; e dentro, di nuovo, $YG \equiv_\beta G(YG)$. Y «srotola» la definizione una chiamata alla volta, senza usare il nome della funzione. Il caso che ferma la ricorsione lo deve mettere il programmatore.

Esempio: fattoriale (supponendo di avere numeri, if, $n = 0$, $n - 1$, $*$).

$$G = \lambda f.\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * f(n - 1)$$

Calcolo $YG1$. Per brevità $W = (\lambda x.G(xx))(\lambda x.G(xx))$, il termine che si ricopia:

$$\begin{aligned}
YG1 &\to W1 \\
&\to GW1 \\
&\to (\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * W(n - 1))1 \\
&\to \text{if}(1 = 0)\ \text{then}\ 1\ \text{else}\ 1 * W(1 - 1) \\
&\xrightarrow{*} 1 * W0 \\
&\to 1 * GW0 \\
&\to 1 * (\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * W(n - 1))0 \\
&\to 1 * (\text{if}(0 = 0)\ \text{then}\ 1\ \text{else}\ 0 * W(0 - 1)) \\
&\xrightarrow{*} 1 * 1 = 1
\end{aligned}$$

Nell'ultimo if la condizione è vera: $W(0 - 1)$ non viene mai calcolato e la ricorsione si ferma.

> Y funziona con la **call-by-name**, che non valuta subito gli argomenti: in $F(xx)$ l'argomento xx viene ridotto solo se e quando serve nel corpo di F. Con la call-by-value xx verrebbe valutato subito, e sarebbe un loop infinito.

Per questo in JavaScript, che è call-by-value, Y è un po' diverso. `x(x)` diventa `y => x(x)(y)`: non è più un valore da calcolare subito ma una funzione che aspetta ancora l'argomento `y`, e la ricorsione parte solo quando `f` viene davvero chiamata.

```
const Y = f => (x => x(x))(x => f(y => x(x)(y)));
const factorial = f => (x => (x === 1 ? 1 : x * f(x - 1)));
const YFactorial = Y(factorial)(10);
```

1.8 Codifiche: booleani e numeri

Il λ-calcolo puro ha solo funzioni. Booleani, condizionali e numeri (quello che serve per essere Turing-equivalenti) si ottengono con le **codifiche**. L'idea: non guardare cosa un valore **rappresenta** ma cosa ci si può **fare**, e descrivere quell'uso con una funzione.

Con un **booleano** si fa una scelta fra due alternative. Quindi un booleano è una funzione che, date due scelte, ne seleziona una:

$\text{TRUE} = \lambda t.\lambda f.t$	$\text{TRUE}\,ab \Rightarrow a$
$\text{FALSE} = \lambda t.\lambda f.f$	$\text{FALSE}\,ab \Rightarrow b$

Da qui gli operatori logici: NOT applica il booleano b alle due scelte FALSE e TRUE, così TRUE sceglie FALSE e viceversa.

$$\text{NOT} = \lambda b.b\,\text{FALSE}\,\text{TRUE}$$

$$\text{NOT TRUE} \to \text{TRUE FALSE TRUE} = (\lambda t.\lambda f.t)\,\text{FALSE TRUE} \Rightarrow \text{FALSE}$$
$$\text{NOT FALSE} \to \text{FALSE FALSE TRUE} = (\lambda t.\lambda f.f)\,\text{FALSE TRUE} \Rightarrow \text{TRUE}$$

Con la stessa idea, AND e OR usano il primo booleano b per scegliere. AND: se b è vero il risultato è c, altrimenti FALSE. OR: se b è vero il risultato è TRUE, altrimenti c.

$$\text{AND} = \lambda b.\lambda c.bc\,\text{FALSE} \qquad \text{OR} = \lambda b.\lambda c.b\,\text{TRUE}\,c$$

$$\begin{aligned}\text{AND TRUE TRUE} &\Rightarrow \text{TRUE TRUE FALSE} = (\lambda t.\lambda f.t)\,\text{TRUE FALSE}\\ &\to (\lambda f.\,\text{TRUE})\,\text{FALSE} \to \text{TRUE}\\ \text{OR TRUE FALSE} &\Rightarrow \text{TRUE TRUE FALSE} = (\lambda t.\lambda f.t)\,\text{TRUE FALSE}\\ &\to (\lambda f.\,\text{TRUE})\,\text{FALSE} \to \text{TRUE}\end{aligned}$$

Il **condizionale** sceglie in base al valore della condizione: passa i due rami al booleano c.

$$\text{IF} = \lambda c.\lambda\ \text{then}.\lambda\ \text{else}.c\ \text{then else} \qquad \text{IF TRUE}\,ab \Rightarrow a \qquad \text{IF FALSE}\,ab \Rightarrow b$$

$$\begin{aligned}\text{IF TRUE}\,e_1e_2 &= (\lambda c.\lambda\ \text{then}.\lambda\ \text{else}.c\ \text{then else})\,\text{TRUE}\,e_1e_2\\ &\to (\lambda\ \text{then}.\lambda\ \text{else}.\,\text{TRUE then else})e_1e_2\\ &\to (\lambda\ \text{else}.\,\text{TRUE}\,e_1\ \text{else})e_2\\ &\to \text{TRUE}\,e_1e_2 = (\lambda t.\lambda f.t)e_1e_2\\ &\to (\lambda f.e_1)e_2 \to e_1\end{aligned}$$

> **Esercizio 7**
> Calcolare $\text{IF FALSE}\,e_1e_2$.

I **numeri naturali** hanno una definizione induttiva: zero è un naturale; se n è un naturale, anche $\text{Succ}\ n$ (il successore) lo è. Le operazioni si definiscono allo stesso modo:

	Caso zero	**Caso successore**
Somma	$m + 0 = m$	$m + \text{Succ}\ n = \text{Succ}(m + n)$
Prodotto	$m \times 0 = 0$	$m \times \text{Succ}\ n = m \times n + m$

Nel λ-calcolo un numero si codifica con **cosa ci si fa**: ripetere qualcosa n volte. Il **numerale di Church** C_n prende una funzione s (il successore) e un valore iniziale z (lo zero), e applica s a z per n volte. Il numero 3 vuol dire «fai una cosa tre volte».

$$C_0 = \lambda s.\lambda z.z \qquad C_1 = \lambda s.\lambda z.sz \qquad C_2 = \lambda s.\lambda z.s(sz)$$
$$C_3 = \lambda s.\lambda z.s(s(sz)) \qquad C_n = \lambda s.\lambda z.s^n z$$

Le operazioni sui numerali:

Definizione	**Come funziona**
$\text{SUCC} = \lambda n.\lambda s.\lambda z.s(nsz)$	nsz applica s a z per n volte; poi una s in più: in tutto $n+1$ volte. $\text{SUCC}\, C_1 \Rightarrow C_2$
$\text{PLUS} = \lambda m.\lambda n.\lambda s.\lambda z.ms(nsz)$ oppure $\lambda m.\lambda n.m\,\text{SUCC}\,n$	nsz calcola n; $ms(\ldots)$ ci applica s altre m volte. Nella seconda forma: applico SUCC a n per m volte. $\text{PLUS}\, C_i C_j \Rightarrow \text{SUCC}^i(C_j)$
$\text{TIMES} = \lambda m.\lambda n.m(\text{PLUS}\,n)C_0$	m conta le ripetizioni: partendo da C_0, sommo n per m volte. $\text{TIMES}\, C_i C_j \Rightarrow \text{PLUS}\, C_j(\text{PLUS}\, C_j(\ldots(\text{PLUS}\, C_j C_0)))$ con i somme

$\text{SUCC}\, C_1$ passo per passo. Per non confondere le variabili, in C_1 rinomino s, z in s', z'; il secondo passo riduce dentro il corpo della funzione (riduzione standard):

$$\begin{aligned}\text{SUCC}\, C_1 = (\lambda n.\lambda s.\lambda z.s(nsz))C_1 &\to \lambda s.\lambda z.s(C_1 sz) \\ &= \lambda s.\lambda z.s((\lambda s'.\lambda z'.s'z')sz) \\ &\Rightarrow \lambda s.\lambda z.s(sz) = C_2\end{aligned}$$

Esercizio 8
Definire ISZERO in modo che $\text{ISZERO}\, C_0 = \text{TRUE}$ e $\text{ISZERO}\, C_i = \text{FALSE}$ per $i \neq 0$; calcolare $\text{ISZERO}\, C_1$.

Esercizio 9
a. Calcolare $\text{PLUS}\, C_1 C_1$. Poi dimostrare:
b. $\text{IF}(\text{IF FALSE TRUE FALSE})C_1 C_2 \equiv_\beta C_2$
c. $\text{PLUS}\, C_2 C_1 \equiv_\beta C_3$
d. $\text{TIMES}\, C_2 C_2 \equiv_\beta C_4$

Quindi il λ-calcolo è **Turing completo**: con codifiche furbe rappresenta quasi ogni costrutto dei linguaggi. Ma i programmi sarebbero lenti (migliaia di chiamate di funzione), lunghi (centinaia di righe) e difficili da capire. In pratica si usano linguaggi più ricchi, che hanno già le primitive giuste (numeri, booleani, `if`).

2 Controllo dei tipi

Cosa devi saper fare

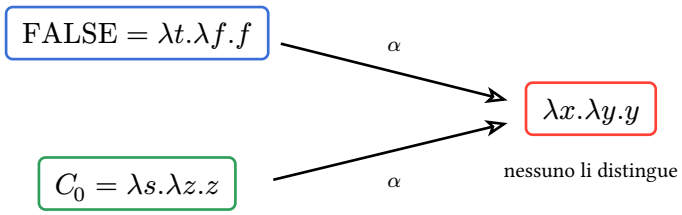
- spiegare a cosa servono i tipi e cosa vuol dire type safety
- leggere una regola di inferenza e costruire un albero di derivazione
- dimostrare una proprietà per induzione sulle derivazioni
- valutare passo per passo un'espressione con le regole small-step, dicendo il nome di ogni regola
- dire se un'espressione è ben tipata costruendo l'albero, o mostrare perché l'albero non si può costruire
- enunciare progresso e conservazione e dimostrarli per il caso dell'if

2.1 Perché servono i tipi

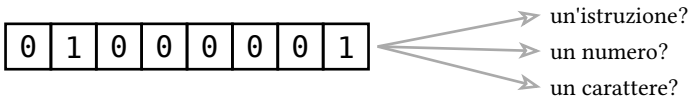
Nel λ-calcolo programmi e valori sono tutti funzioni. Quindi niente impedisce di usare un valore nel posto sbagliato:

$$\mathrm{FALSE}\, C_0 = (\lambda t.\lambda f.f)(\lambda s.\lambda z.z) \to \lambda f.f$$

FALSE si può davvero applicare a C_0, e produce un valore. Ma non ha senso: FALSE serve a scegliere fra due alternative, non a ricevere un numero. Peggio: C_0 e FALSE sono **lo stesso termine**, a meno di α-conversione. Lo stesso vale per $\mathrm{IF}\, C_0 e_1 e_2$ o $\mathrm{TRUE}\, C_0$.



Non è solo un problema del λ-calcolo. Nel **linguaggio macchina** le istruzioni sono parole della macchina (sequenze di bit), i dati sono parole della macchina, e le operazioni prendono sequenze di bit e restituiscono sequenze di bit.



Un principio centrale dell'ingegneria del software è **trovare presto gli errori di programmazione** (come $\mathrm{FALSE}\, C_0$). I linguaggi lo fanno con i **sistemi di tipo** (e più in generale con gli strumenti di **analisi statica**, che esaminano il programma senza eseguirlo).

I **tipi** raggruppano i valori in base al comportamento che vorremmo avessero. Il tipo di un dato dice come il linguaggio permette di usarlo. Un tipo:

- **limita i valori** che un'espressione (una variabile, una funzione) può assumere;
- **definisce le operazioni** ammesse su quei valori e come vengono memorizzati.

I linguaggi hanno dei tipi di base: interi (di varie dimensioni), numeri in virgola mobile (approssimano i reali), caratteri, booleani. $e : T$ si legge «e ha tipo T»:

Espressione	Cosa dice
$3 + 2 : \mathrm{Int}$ ✓	il risultato è un intero, e $+$ è usato bene: riceve due interi
$\mathrm{true} : \mathrm{Int}$ ✗	true non è un intero
$2 + \mathrm{true}$ ✗	$+$ non è applicato bene

Perché controllare i tipi? In un programma grande c'è questo pezzo:

```
if (condizione_complicata)
  { return "hello"/10; }
```

Se i test non rendono mai vera `condizione_complicata`, la divisione di una stringa nel ramo `then` non viene mai scoperta. Poi un input non previsto dai test fa eseguire il ramo, e il programma va in errore **durante l'esecuzione**. Bastava segnalare `"hello"/10` come espressione non ammessa.

Lo stesso codice in JavaScript e in TypeScript:

```
function addNumbers(x, y) {
  return x + y; }
console.log(addNumbers(3, "0"));
```

JavaScript stampa `"30"`, una stringa. Il `+` accetta anche una coppia (numero, stringa): il numero viene trasformato in stringa (**coercion**) e le due stringhe attaccate.

```
function addNumbers(x: number,
                    y: number) {
  return x + y; }
console.log(addNumbers(3, "0"));
```

TypeScript si accorge del bug e dà un errore di tipo: *Argument of type "string" is not assignable to parameter of type "number"*. Come? Con delle **regole di tipo**.

2.2 Sistemi di tipi

«Well-typed programs cannot go wrong» — Robin Milner

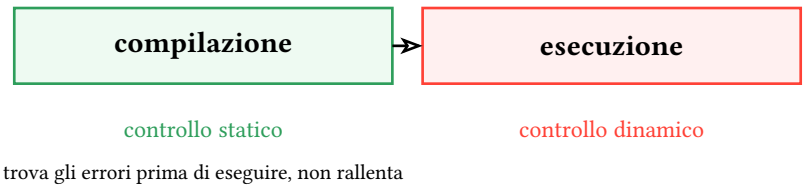
Un **sistema di tipi** associa tipi ai valori calcolati. Seguendo il flusso dei valori, prova a **dimostrare** che non avvengano errori di tipo. È il sistema stesso a stabilire cos'è un errore di tipo: un'operazione che si aspetta un certo tipo di valore usata con un valore per cui non ha senso.

Più precisamente è un metodo **sintattico**, **effettivo** e **strutturale** per dimostrare che il programma non ha comportamenti anomali:

Metodo	Cosa vuol dire
sintattico	l'analisi segue la struttura sintattica del programma
effettivo	c'è un algoritmo che controlla i vincoli sui tipi, da mettere in un compilatore o in un interprete
strutturale	il tipo di un'espressione dipende solo dai tipi delle sue sottoespressioni (**composizionale**)

Type safety (uso corretto dei tipi): se un'espressione ha un tipo, il tipo garantisce che venga usata nel modo giusto. Senza type safety si scrivono programmi pieni di bug, e i bug sono **vulnerabilità**: un attaccante può sfruttarli per cambiare il comportamento del programma o prenderne il controllo.

Il controllo può essere **statico**, **dinamico** o un misto dei due:



Il **type checker** (controllore dei tipi) verifica che il programma rispetti le intenzioni del programmatore, scritte nelle **annotazioni di tipo** (come `x: number`). Se un programma supera il controllo, durante l'esecuzione **non applicherà mai un'operazione a un valore del tipo sbagliato**. È una **correttezza parziale**: il programma è corretto rispetto ai tipi, non rispetto ad altri errori.

Resta una domanda: come si fa a sapere che i programmi che superano il controllo si comportano davvero bene durante l'esecuzione? Servono due cose:

1. un modo per **specificare** un sistema di tipi;
2. un modo per **dimostrare** che è corretto. Per questo bisogna capire nel dettaglio cosa succede quando un programma viene eseguito, cioè il suo **significato**.

Lo strumento per entrambe sono le regole di inferenza.

2.3 Regole di inferenza

Le abbiamo già usate per le strategie di valutazione: sopra la riga le **premesse**, sotto la **conclusione**, a destra il **nome** della regola. Se valgono tutte le premesse, si può concludere q. Una regola senza premesse è un **assioma**: q è un fatto accettato dal sistema.

$$\frac{p_1 \quad p_2 \quad \dots \quad p_n}{q}\ \text{(nome)} \qquad \frac{}{q}\ \text{(assioma)}$$

Una conclusione è valida **rispetto alle regole del sistema scelto**. Esempio: le regole per l'uguaglianza fra termini, riflessiva e transitiva:

$$\frac{}{t = t}\ \text{(Rifl)} \qquad \frac{t_1 = t_2 \quad t_2 = t_3}{t_1 = t_3}\ \text{(Trans)}$$

Una dimostrazione mette insieme più regole in un **albero di derivazione**: le foglie in alto (premesse o assiomi), ogni riga è una regola applicata, la radice in basso è la conclusione. Dalle premesse $a = b$, $b = c$, $c = d$ si deriva $a = d$:

$$\dfrac{\dfrac{\dfrac{a = b \quad b = c}{a = c}\ \text{(Trans)} \quad c = d}{a = d}\ \text{(Trans)}}{}$$

Le regole possono anche **definire un insieme**: le regole generano gli elementi, una derivazione dimostra che un elemento ci sta dentro. Per esempio i pari:

$$\{0\} \subseteq \text{Pari} \Rightarrow \{0, 2\} \subseteq \text{Pari} \Rightarrow \{0, 2, 4\} \subseteq \text{Pari} \Rightarrow \dots$$

$$\text{Pari} = \{0, 2, 4, 6, \dots\}$$

$$\frac{}{0 \in \text{Pari}}\ \text{(Zero)}$$

$$\frac{(n-2) \in \text{Pari}}{n \in \text{Pari}}\ \text{(Passo, } n \geq 2\text{)}$$

Derivazione di $4 \in \text{Pari}$:

$$\dfrac{\dfrac{\dfrac{}{0 \in \text{Pari}}\ \text{(Zero)}}{2 \in \text{Pari}}\ \text{(Passo)}}{4 \in \text{Pari}}\ \text{(Passo)}$$

Per dimostrare una proprietà di **tutti** gli elementi generati si segue la struttura delle regole: è l'**induzione sulle derivazioni**. Un caso per ogni regola: gli assiomi sono i casi base, le altre regole i passi induttivi.

Proprietà da dimostrare: $P(n) \equiv n \bmod 2 = 0$.

Caso base: regola (Zero)	**Passo induttivo: regola (Passo)**
$P(0)$ vale perché $0 \bmod 2 = 0$.	Suppongo $P(n-2)$ e dimostro $P(n)$. $(n-2) \bmod 2 = 0$ vuol dire che esiste k con $n - 2 = 2k$. Allora $n = 2k + 2 = 2(k+1)$: con $k' = k + 1$ ho $n = 2k'$, quindi $n \bmod 2 = 0$.

Conclusione: per ogni $n \in \text{Pari}$ vale $n \bmod 2 = 0$.

2.4 Un linguaggio di espressioni

Primo caso di studio: un sistema di tipi per un piccolo linguaggio di espressioni con booleani e numeri. Prima la **sintassi**:

Espressioni

$$
\begin{aligned}
E ::= \ & \text{true} \\
\mid \ & \text{false} \\
\mid \ & \text{NV} \\
\mid \ & \text{if } E \text{ then } E \text{ else } E \\
\mid \ & \text{succ } E \\
\mid \ & \text{pred } E \\
\mid \ & \text{isZero } E
\end{aligned}
$$

Valori

$$
\begin{aligned}
V ::= \ & \text{true} \\
\mid \ & \text{false} \\
\mid \ & \text{NV}
\end{aligned}
$$

Valori numerici

$$\text{NV} ::= 0 \mid 1 \mid 2 \mid \ldots$$

I **valori** sono i risultati finali: le espressioni che non vanno più calcolate.

Poi la **semantica**, cioè cosa succede quando un'espressione viene eseguita: un **interprete** scritto con regole di inferenza. Ogni regola dice come fare **un passo** di calcolo $E \to E'$ (semantica *small-step*, a piccoli passi).

Condizionale

$$\text{if true then } E_1 \text{ else } E_2 \to E_1 \quad \text{(If-True)} \qquad \text{if false then } E_1 \text{ else } E_2 \to E_2 \quad \text{(If-False)}$$

$$\frac{E \to E'}{\text{if } E \text{ then } E_1 \text{ else } E_2 \to \text{if } E' \text{ then } E_1 \text{ else } E_2} \quad \text{(If-Cond)}$$

If-Cond ha la stessa forma delle regole delle strategie: finché la condizione non è un valore, si calcola la condizione. Quando diventa true o false scatta If-True o If-False.

Operazioni sui numeri

$$\frac{E \to E'}{\text{succ } E \to \text{succ } E'} \quad \text{(Succ)} \qquad \frac{m = n + 1}{\text{succ } n \to m} \quad \text{(SuccNat)}$$

$$\frac{E \to E'}{\text{pred } E \to \text{pred } E'} \quad \text{(Pred)} \qquad \text{pred } 0 \to 0 \quad \text{(PredZero)} \qquad \frac{n > 0 \quad m = n - 1}{\text{pred } n \to m} \quad \text{(PredNat)}$$

$$\frac{E \to E'}{\text{isZero } E \to \text{isZero } E'} \quad \text{(IsZero)} \qquad \text{isZero } 0 \to \text{true} \quad \text{(IsZeroZero)} \qquad \frac{n > 0}{\text{isZero } n \to \text{false}} \quad \text{(IsZeroSucc)}$$

Lo schema è sempre lo stesso: la regola a sinistra calcola l'argomento finché diventa un numero n, le altre fanno l'operazione. pred 0 dà 0: non ci sono numeri negativi.

Passo dopo passo, `if isZero 0 then 2 else pred 1` si calcola così:

$$\frac{\text{isZero } 0 \to \text{true}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1 \to \text{if true then } 2 \text{ else pred } 1} \quad \text{(If-Cond)}$$

$$\text{if true then } 2 \text{ else pred } 1 \to 2 \quad \text{(If-True)}$$

Finora l'interprete sa solo calcolare. Ora gli aggiungiamo i **tipi**: solo due, Bool e Nat (i numeri).

$$T ::= \text{Bool} \mid \text{Nat}$$

Il controllo dei tipi è una relazione fra espressioni e tipi, scritta $E : T$ («E ha tipo T»). È definita da regole di inferenza, una per ogni forma sintattica: il metodo è **sintattico**, per induzione sulla struttura del programma.

Regole di tipo

$$\frac{}{\text{true} : \text{Bool}}\ (\text{T-True}) \qquad \frac{}{\text{false} : \text{Bool}}\ (\text{T-False}) \qquad \frac{}{n : \text{Nat}}\ (\text{T-Nat})$$

$$\frac{E : \text{Bool} \quad E_1 : T \quad E_2 : T}{\text{if } E \text{ then } E_1 \text{ else } E_2 : T}\ (\text{T-If})$$

$$\frac{E : \text{Nat}}{\text{succ } E : \text{Nat}}\ (\text{T-Succ}) \qquad \frac{E : \text{Nat}}{\text{pred } E : \text{Nat}}\ (\text{T-Pred}) \qquad \frac{E : \text{Nat}}{\text{isZero } E : \text{Bool}}\ (\text{T-IsZero})$$

La relazione di tipo è la **più piccola** relazione che soddisfa tutte le istanze di queste regole. Un'espressione E è **ben tipata** (*well-typed*) se esiste un T con $E : T$; altrimenti è *ill-typed*.

Controllo («E ha tipo T?»): dati E e T si cerca un **albero di derivazione** fatto di istanze delle regole. Per esempio:

$$\frac{\dfrac{}{0 : \text{Nat}}\ (\text{T-Nat}) \quad \dfrac{0 : \text{Nat}}{\text{isZero } 0 : \text{Bool}}\ (\text{T-IsZero}) \quad \dfrac{}{2 : \text{Nat}}\ (\text{T-Nat}) \quad \dfrac{1 : \text{Nat}}{\text{pred } 1 : \text{Nat}}\ (\text{T-Pred})}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1 : \text{Nat}}\ (\text{T-If})$$

Inferenza: è il problema opposto, data solo E si cerca un T (e la derivazione).

Le regole sono **deterministiche**: per ogni forma sintattica ce n'è una sola, quindi si trasformano subito in un algoritmo che calcola il tipo guardando i tipi delle sottoespressioni:

```
typeof(E) =
  E = true | false          →  Bool
  E = n                     →  Nat
  E = if E1 then E2 else E3
      →  T1 = typeof(E1), T2 = typeof(E2), T3 = typeof(E3);
         se T1 = Bool e T2 = T3 allora T2, altrimenti errore
  E = succ E1 | pred E1     →  se typeof(E1) = Nat allora Nat, altrimenti errore
  E = isZero E1             →  se typeof(E1) = Nat allora Bool, altrimenti errore
```

Il tipo di un'espressione dipende solo dai tipi delle sue parti: è la **composizionalità**. Per questo T-If vuole i due rami dello **stesso** tipo T, che diventa il tipo di tutto l'`if`: chi usa quell'`if` deve sapere un tipo preciso. Per esempio l'argomento di pred deve essere Nat e la condizione di un `if` deve essere Bool.

> I sistemi di tipo sono **imprecisi**: danno un'approssimazione prudente che considera tutte le esecuzioni possibili, e anche qualcuna impossibile. if true then 0 else false dà sicuramente un numero, ma non ha tipo: T-If considera anche il ramo `else`, che a runtime non verrebbe mai eseguito, e Nat $\neq$ Bool.

Esercizio 10
Per ogni espressione: dire se è ben tipata costruendo l'albero di derivazione (e dare il tipo), oppure mostrare che l'albero non si può costruire. Poi valutarla passo per passo fino a un valore, o fino a un'espressione che non si riduce più. a. if isZero(pred 0) then true else false
b. if isZero(pred true) then true else false
c. isZero(succ(pred 5))
d. pred(pred(pred 2))
e. if true then isZero 2 else succ 1

2.5 Type safety

Un sistema di tipi è **corretto** (*type safe*, *sound*) se vale ciò che promette Milner: i programmi ben tipati non «vanno storti». Si traduce in due proprietà, da dimostrare entrambe:

Proprietà	Cosa dice
Progresso: se $E : T$ allora E è un valore oppure $E \to E'$ per qualche E'	un'espressione ben tipata **non si blocca** durante l'esecuzione: o ha finito, o può fare un passo
Conservazione: se $E : T$ e $E \to E'$ allora $E' : T$	le regole di esecuzione **preservano il tipo**: dopo un passo l'espressione resta ben tipata, con lo stesso tipo

Insieme garantiscono che l'esecuzione di un'espressione ben tipata non si blocca mai su un tipo sbagliato.

> Senza tipi può succedere che l'espressione non sia un valore e non esista nessuna regola per proseguire: if 0 then 2 else 1 resta bloccata, perché la condizione non è né true né false. Il progresso dice che con i tipi non capita.

Per la conservazione, esempio sull'albero visto prima: if isZero 0 then 2 else pred 1 : Nat fa un passo e diventa if true then 2 else pred 1, che ha ancora tipo Nat (T-True, 2 : Nat e pred 1 : Nat danno T-If). Poi diventa 2 : Nat.

Le dimostrazioni sono per **induzione sulla derivazione di $E : T$**: un caso per ogni regola di tipo. Si mostra il caso dell'`if` (gli altri sono analoghi).

Progresso. Casi base (T-True, T-False, T-Nat): E è già un valore. Caso $E =$ if E_1 then E_2 else E_3: per T-If vale E_1 : Bool. Per ipotesi induttiva (a) E_1 è un valore, quindi true o false, e scatta If-True o If-False; oppure (b) $E_1 \to E_{1'}$, e scatta If-Cond. In ogni caso E fa un passo.

Conservazione. Casi base: E è un valore, non si riduce, la tesi è vera. Caso $E =$ if E_1 then E_2 else E_3 : T: per T-If vale E_1 : Bool, $E_2 : T$, $E_3 : T$. Il passo $E \to E'$ può venire da tre regole:
1. If-True: $E' = E_2$, che ha tipo T;
2. If-False: $E' = E_3$, che ha tipo T;
3. If-Cond con $E_1 \to E_{1'}$: per ipotesi induttiva $E_{1'}$: Bool, quindi con T-If anche if $E_{1'}$ then E_2 else E_3 : T.

3 Il lambda calcolo tipato

Cosa devi saper fare

- usare l'ambiente dei tipi Γ e il giudizio $\Gamma \vdash e : \tau$
- costruire l'albero di tipo di un termine di FUN con T-Var, T-Fun, T-App, T-If, o mostrare che non esiste
- valutare un termine con la call-by-value, fino a un valore o a un termine bloccato
- enunciare progresso, conservazione e lemma di sostituzione, e dire dove serve il lemma
- spiegare perché Ω e Y non sono tipabili e come fix recupera la ricorsione

3.1 Tipi e regole di tipo

Il λ-calcolo «puro» non ha tipi. Qui lo rendiamo tipato, partendo da un solo tipo di base, i booleani, più il tipo delle funzioni:

$$\tau ::= \text{Bool} \mid \tau \to \tau$$

Esempi di tipi: Bool, Bool → Bool, (Bool → Bool) → Bool. La freccia **associa a destra**: Bool → Bool → Bool sta per Bool → (Bool → Bool). Un tipo $\tau_1 \to \tau_2$ è quello delle funzioni che prendono un argomento di tipo τ_1 e restituiscono un risultato di tipo τ_2.

Il linguaggio, che chiamiamo **FUN**, ha il parametro con il suo tipo (nel formato tradizionale: $\lambda x : \tau . e$):

Espressioni

$e ::= x$ variabile
$\mid \text{fun}\, x : \tau = e$ funzione
$\mid \text{Apply}(e, e)$ applicazione
$\mid \text{true} \mid \text{false}$
$\mid \text{if}\, e\, \text{then}\, e\, \text{else}\, e$

Valori

$v ::= \text{fun}\, x : \tau = e$
$\mid \text{true} \mid \text{false}$

Il condizionale è un'**espressione**, non un comando: i due rami contengono espressioni.

La semantica è quella già vista, riscritta con la nuova sintassi (sostituzione che evita la cattura). If-Cond si trova anche con il nome IF-Eval. Per l'applicazione si usa **call-by-value**: prima si calcola la funzione, poi l'argomento, poi si fa la β-riduzione.

Semantica

$$\text{Apply}(\text{fun}\, x : \tau = e_1, v_2) \to e_1\{x := v_2\} \quad \text{(App)}$$

$$\frac{e_1 \to e_{1'}}{\text{Apply}(e_1, e_2) \to \text{Apply}(e_{1'}, e_2)}\ \text{(App-1)} \qquad \frac{e_2 \to e_{2'}}{\text{Apply}(v_1, e_2) \to \text{Apply}(v_1, e_{2'})}\ \text{(App-2)}$$

$$\frac{e \to e'}{\text{if}\, e\, \text{then}\, e_1\, \text{else}\, e_2 \to \text{if}\, e'\, \text{then}\, e_1\, \text{else}\, e_2}\ \text{(If-Cond)}$$

$$\text{if true then}\, e_1\, \text{else}\, e_2 \to e_1 \quad \text{(If-True)} \qquad \text{if false then}\, e_1\, \text{else}\, e_2 \to e_2 \quad \text{(If-False)}$$

Il problema di tipare le funzioni. Per dare un tipo a $\text{fun}\, x : \tau_1 = e$ ci aspettiamo $\tau_1 \to \tau_2$, dove τ_2 è il tipo del corpo e. Ma il tipo di e dipende dal tipo di x, che si decide nella funzione. Serve ricordare i tipi delle variabili: l'**ambiente dei tipi** Γ, una funzione che associa nomi a tipi.

$$\Gamma = x_1 : \tau_1, x_2 : \tau_2, \ldots, x_k : \tau_k \quad \text{significa} \quad \Gamma(x_i) = \tau_i$$

$\Gamma, x : \tau$ è Γ estesa con $x : \tau$: vale τ per x, e $\Gamma(y)$ per ogni $y \neq x$. L'ambiente vuoto $\varnothing$ non lega nessuna variabile. Esempio con $\Gamma = x : \tau, y : \tau'$:

$\Gamma(x)$	$\Gamma(z)$	$(\Gamma, z : \tau_2)(z)$
τ	indefinito	τ_2

Il controllo dei tipi diventa una relazione a **tre posti**, il **giudizio di tipo**:

$$\Gamma \vdash e : \tau$$

«nell'ambiente Γ, l'espressione e ha tipo τ». Nel linguaggio dei compilatori l'ambiente dei tipi è la **tabella dei simboli** (*symbol table*); le regole le applica il compilatore durante l'analisi statica.

Regole di tipo di FUN

$$\frac{}{\Gamma \vdash \text{true} : \text{Bool}}\ \text{(T-True)} \qquad \frac{}{\Gamma \vdash \text{false} : \text{Bool}}\ \text{(T-False)} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \text{(T-Var)}$$

$$\frac{\Gamma \vdash e : \text{Bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau}\ \text{(T-If)}$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x : \tau_1 = e : \tau_1 \to \tau_2}\ \text{(T-Fun)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}\ \text{(T-App)}$$

- **T-Var**: una variabile ha il tipo che le dà l'ambiente.
- **T-If**: i rami hanno lo stesso tipo τ, che può essere anche un tipo funzione: si possono avere condizionali i cui rami sono funzioni.
- **T-Fun**: il parametro x è dichiarato con il suo tipo τ_1; per tipare il corpo si usa $\Gamma, x : \tau_1$. Questa dichiarazione **sovrascrive** eventuali dichiarazioni precedenti di x e vale solo nel corpo (è il suo *scope*). Il risultato è il tipo del corpo.
- **T-App**: se e_1 trasforma argomenti di tipo τ_1 in risultati di tipo τ_2 e e_2 ha tipo τ_1, il risultato è di tipo τ_2.

Esempio: $\varnothing \vdash \text{fun } x : \text{Bool} = x : \text{Bool} \to \text{Bool}$, perché $\varnothing, x : \text{Bool} \vdash x : \text{Bool}$.

Esempio più grande: $\lambda f.f(\text{if false then true else false})$, cioè $\text{fun } f : \text{Bool} \to \text{Bool} = \text{Apply}(f, e_0)$ con $e_0 = \text{if false then true else false}$. Sia $\Gamma_f = \varnothing, f : \text{Bool} \to \text{Bool}$.

$$\dfrac{\dfrac{\dfrac{}{\Gamma_f \vdash f : B \to B}\ \text{(T-Var)} \quad \dfrac{\Gamma_f \vdash \text{false} : B \quad \Gamma_f \vdash \text{true} : B \quad \Gamma_f \vdash \text{false} : B}{\Gamma_f \vdash e_0 : B}\ \text{(T-If)}}{\Gamma_f \vdash \text{Apply}(f, e_0) : B}\ \text{(T-App)}}{\varnothing \vdash \text{fun } f : B \to B = \text{Apply}(f, e_0) : (B \to B) \to B}\ \text{(T-Fun)}$$

Esercizio 11
Per ogni termine: dire se è ben tipato costruendo l'albero di derivazione (e dare il tipo), oppure mostrare che l'albero non si può costruire. Poi valutarlo fino a un valore, o fino a un termine che non si riduce più. a. $\text{Apply}(\text{fun } x : \text{Bool} = x, \text{true})$
b. $\text{Apply}(\text{fun } x : \text{Bool} = \text{Apply}(x, \text{true}), \text{false})$
c. $\text{fun } f : \text{Bool} \to \text{Bool} = \text{Apply}(f, \text{true})$
d. $\text{Apply}(\text{fun } f : \text{Bool} \to \text{Bool} = (\text{fun } x : \text{Bool} = \text{Apply}(f, x)), \text{fun } x : \text{Bool} = x)$

3.2 Type safety nel λ-calcolo tipato

Le due proprietà valgono anche qui, con una condizione sul progresso: l'espressione deve essere **chiusa** (nessuna variabile libera), cioè tipata nell'ambiente vuoto.

Proprietà	Cosa dice
Progresso: se $\varnothing \vdash e : \tau$ allora e è un valore oppure $e \to e'$	un'espressione chiusa e ben tipata non si blocca
Conservazione: se $\Gamma \vdash e : \tau$ e $e \to e'$ allora $\Gamma \vdash e' : \tau$	un passo di esecuzione preserva il tipo

Si dimostrano ancora per induzione sulle derivazioni di tipo.

Progresso. Costanti booleane: come prima. Variabili: caso banale, perché nell'ambiente vuoto una variabile non ha tipo. Astrazione: è già un valore. Applicazione $e = \text{Apply}(e_1, e_2)$ con $\varnothing \vdash e_1 : \tau_1 \to \tau_2$ e $\varnothing \vdash e_2 : \tau_1$: per ipotesi induttiva e_1 e e_2 sono valori o possono fare un passo.

- Se e_1 fa un passo, e fa un passo (prima regola dell'applicazione).
- Se e_1 è un valore e e_2 fa un passo, e fa un passo (seconda regola).
- Se sono entrambi valori, e_1 ha tipo funzione, quindi è $\text{fun}\, x : \tau_1 = e_3$, e e fa un passo con la β-riduzione.

Conservazione. Il caso difficile è l'applicazione $e = \text{Apply}(\text{fun}\, x : \tau_1 = e_3, e_2)$ con e_2 valore, che si riduce a $e' = e_3\{x := e_2\}$. Dal tipo di e si ricava $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ e $\Gamma \vdash e_2 : \tau_1$, e bisogna dimostrare $\Gamma \vdash e_3\{x := e_2\} : \tau_2$. Ma abbiamo tipato e_3 e non $e_3\{x := e_2\}$: ci vuole un lemma che dica che la sostituzione non cambia il tipo.

Lemma di sostituzione. Se $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ e $\Gamma \vdash e_2 : \tau_1$, allora $\Gamma \vdash e_3\{x := e_2\} : \tau_2$.

Sostituire il parametro formale con un parametro attuale dello **stesso tipo** conserva il tipo. Esempio: con $x : \text{Bool}$ vale $\text{if}\, x \,\text{then false else true} : \text{Bool}$, quindi anche $(\text{if}\, x \,\text{then false else true})\{x := \text{true}\} = \text{if true then false else true} : \text{Bool}$. Il lemma si dimostra per induzione sulla derivazione di $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$; i casi interessanti sono variabili e astrazioni. Con il lemma la conservazione è dimostrata: è proprio $\Gamma \vdash e' : \tau_2$.

3.3 Estensioni e ricorsione

Si può estendere FUN per farlo diventare un vero linguaggio di programmazione. Con le costanti numeriche n (tipo Nat) e le operazioni binarie $e_1 \oplus e_2$ (con $\oplus \in \{+, -, *, /, \%\}$, di tipo noto $\tau_1 \times \tau_2 \to \tau$):

Si calcola prima l'operando sinistro, poi il destro, poi si fa l'operazione ($\delta_\oplus$ è l'operazione vera, per esempio la somma):

$$\frac{e_1 \to e_{1'}}{e_1 \oplus e_2 \to e_{1'} \oplus e_2}\ \text{(Op-1)} \qquad \frac{e_2 \to e_{2'}}{n_1 \oplus e_2 \to n_1 \oplus e_{2'}}\ \text{(Op-2)} \qquad \frac{\delta_\oplus(n_1, n_2) = n}{n_1 \oplus n_2 \to n}\ \text{(Op-3)}$$

$$\frac{}{\Gamma \vdash n : \text{Nat}}\ \text{(T-Nat)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \oplus : \tau_1 \times \tau_2 \to \tau}{\Gamma \vdash e_1 \oplus e_2 : \tau}\ \text{(T-Op)}$$

Si aggiungono anche le **dichiarazioni locali** $\text{let}\ x = e_1\ \text{in}\ e_2$. Si calcola prima e_1; quando è un valore v lo si sostituisce a x in e_2. Il tipo di x è quello di e_1, e vale solo in e_2:

Dichiarazioni locali

$$\frac{e_1 \to e_{1'}}{\text{let } x = e_1 \text{ in } e_2 \to \text{let } x = e_{1'} \text{ in } e_2} \text{ (Let-1)} \qquad \text{let } x = v \text{ in } e_2 \to e_2\{x := v\} \text{ (Let-2)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (T-Let)}$$

Ricorsione. Nel λ-calcolo tipato semplice tutti i programmi tipabili **terminano**. Quindi $\Omega = (\lambda x.xx)(\lambda x.xx)$ e il combinatore Y **non sono tipabili**. Il motivo è l'auto-applicazione xx:

$$\frac{\Gamma \vdash x : \tau_1 \to \tau_2 \quad \Gamma \vdash x : \tau_1}{\Gamma \vdash \text{Apply}(x, x) : \tau_2} \text{ (T-App)}$$

x ha un solo tipo, quindi servirebbe $\tau_1 = \tau_1 \to \tau_2$, che è impossibile. Per avere la ricorsione bisogna aggiungere al linguaggio un costrutto apposta, tipabile e con le proprietà che vogliamo.

Si parte da un esempio. Il fattoriale è il **punto fisso** della funzione G, che prende l'approssimazione f del fattoriale e ne costruisce una migliore:

$$G = \text{fun } f : \text{Nat} \to \text{Nat} = \text{fun } n : \text{Nat} = \text{if isZero } n \text{ then } 1 \text{ else } n * f(\text{pred } n)$$

$$G : (\text{Nat} \to \text{Nat}) \to \text{Nat} \to \text{Nat}$$

Il costrutto $\text{fix } e$ calcola questo punto fisso, e si scrive $\text{fact} = \text{fix } G$.

Il costrutto fix

$$\frac{e \to e'}{\text{fix } e \to \text{fix } e'} \text{ (Fix)} \qquad \text{fix}(\text{fun } f : \tau = e) \to e\{f := \text{fix}(\text{fun } f : \tau = e)\} \text{ (FixFun)}$$

$$\frac{\Gamma \vdash e : \tau \to \tau}{\Gamma \vdash \text{fix } e : \tau} \text{ (T-Fix)}$$

La regola di valutazione «srotola» una volta la ricorsione: il corpo e viene eseguito con f sostituita da fix di sé stesso, così alla prossima chiamata ricorsiva si srotola ancora. Per $\text{fact } 3 = (\text{fix } G)3$:

$$(\text{fix } G)3 \to (\text{fun } n : \text{Nat} = \text{if isZero } n \text{ then } 1 \text{ else } n * (\text{fix } G)(\text{pred } n))3 \xrightarrow{*} 3 * (\text{fix } G)(\text{pred } 3)$$

Il λ-calcolo tipato semplice con fix si chiama **PCF** (*Programming Computable Functions*), un linguaggio funzionale tipato introdotto da Gordon Plotkin nel 1977.

Per scrivere meno c'è lo zucchero sintattico $\text{let rec } x : \tau = e \text{ in } e'$, che sta per $\text{let } x = \text{fix}(\text{fun } x : \tau = e) \text{ in } e'$. In OCaml ci sono due primitive: `fun` per le funzioni non ricorsive e `let rec` per quelle ricorsive.

```
let plusOne = fun x -> x + 1;;
let rec fact x = if x <= 1 then 1 else x * fact (x - 1)
```

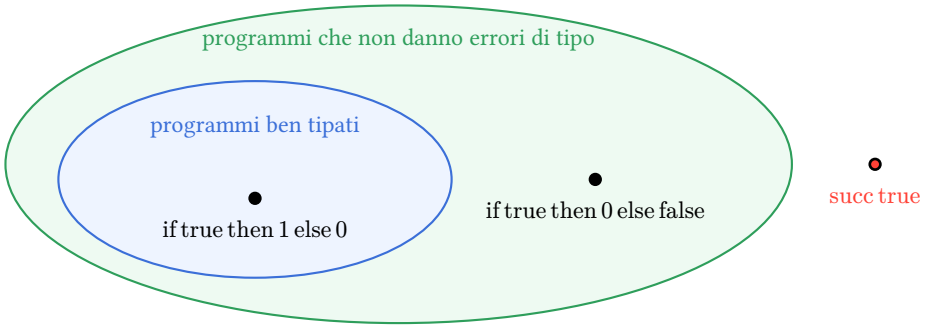
3.4 Cosa possono garantire i tipi

Con PCF il percorso sui tipi è completo, e resta da capire a cosa serve. Il λ-calcolo è un **modello in piccolo** dei linguaggi: sistemi di tipi, terminazione e verifica si studiano qui, dove le regole sono poche, e poi si portano ai linguaggi veri.

Un tipo è una **specifica di comportamento**: $\tau_1 \to \tau_2$ dice che la funzione prende un τ_1 e restituisce un τ_2, e il type checking controlla che il programma rispetti questa specifica. Con tipi più ricchi si controllano anche altre proprietà, per esempio la segretezza e l'autenticità nei protocolli di sicurezza, o l'assenza di deadlock (processi bloccati ad aspettarsi a vicenda) nei sistemi concorrenti.

La garanzia però è solo **parziale**, per un limite che non si può aggirare. Le domande interessanti su un programma (termina su tutti gli input? quanto può crescere la memoria dinamica? lascia uscire informazioni private?) riguardano cosa fa quando gira, e il **teorema di Rice** (1953) dice che **nessuna proprietà non banale del comportamento di un programma è decidibile**: non esiste un algoritmo che risponda sempre giusto («non banale» vuol dire vera per alcuni programmi e falsa per altri). Non basta nemmeno eseguire il programma e guardare, perché non c'è un limite al tempo che può impiegare.

Si esce dal problema **approssimando**: al posto della proprietà esatta se ne controlla una più prudente, che è decidibile. Un sistema di tipi fa proprio questo, e l'approssimazione deve rispettare i comportamenti veri: valgono progresso e conservazione, quindi un programma accettato non dà mai errori di tipo. Il prezzo è che qualche programma corretto viene rifiutato. Nel disegno ogni ovale è un insieme di programmi: i ben tipati stanno tutti dentro quelli senza errori di tipo (è la type safety), ma non li riempiono.



Il punto rosso sta fuori da tutto: succ true si blocca, e il type checker lo rifiuta giustamente. if true then 0 else false invece non dà errori, ma viene rifiutato lo stesso.

Su questa idea sono costruiti gli **analizzatori statici** usati nell'industria, che controllano il codice senza eseguirlo: per esempio Infer, FlowDroid e Roslyn Code Analyzer.

4 OCaml: espressioni, funzioni e tipi

Cosa devi saper fare

- leggere la risposta del toplevel: nome legato, tipo, valore
- tradurre un λ-termine in OCaml e scrivere funzioni con `fun`, `let`, `let ... in`, `let rec`
- distinguere una funzione curried da una che prende una coppia, e usare l'applicazione parziale
- ricavare a mano il tipo di una funzione con il metodo dei vincoli, anche quando è polimorfo (`'a`)

4.1 Il toplevel e i tipi di base

Dopo il modello, il linguaggio. **OCaml** (INRIA, 1996) discende da **ML** (Robin Milner, 1972), il linguaggio funzionale di riferimento: fortemente tipato, con funzioni come valori, variabili immutabili, polimorfismo e pattern matching, e con la **type inference** (il compilatore deduce i tipi da solo). In mezzo c'è **Caml**, che ha aggiunto un'implementazione efficiente e i moduli; OCaml aggiunge gli oggetti. F# di Microsoft è una variante di OCaml. Lo studiamo perché mostra il paradigma funzionale fatto bene, perché molti linguaggi moderni ne stanno copiando dei pezzi, e perché si presta a scrivere interpreti: lo useremo per implementare linguaggi di paradigmi diversi.

Per provare il codice si usa il **toplevel**, un **REPL** (*Read-Eval-Print Loop*): legge un'espressione, la compila, la valuta e stampa tipo e risultato. Si apre con `ocaml` da terminale, oppure con `utop`, con l'interprete online try.ocamlpro.com o con il notebook Jupyter del corso. Il toplevel `ocaml` valuta solo quando trova `;;`, che chiude l'input; nel notebook `;;` serve solo per separare più espressioni nella stessa cella. Negli esempi la risposta del toplevel è scritta come commento, fra `(*` e `*)`.

```
let x = 42;;    (* val x : int = 42 *)
x + 5;;         (* - : int = 47 *)
```

Risposta	Prima di `:`	Fra `:` e `=`	Dopo `=`
`val x : int = 42`	`val x`: è nato il nome `x`	il tipo: `int`	il valore: `42`
`- : int = 47`	`-`: nessun nome nuovo	il tipo: `int`	il valore: `47`

`let` crea un **legame** (*binding*) fra un nome e un valore: **non è un assegnamento**, il valore di `x` non cambierà più. Il tipo è dedotto guardando la **struttura** dell'espressione, senza calcolarla.

I nomi contengono lettere, cifre, `_` e `'`, e iniziano con una lettera minuscola o con `_`: `x_plus_3` e `x'` vanno bene; `Seven` no (con la maiuscola OCaml cerca un **costruttore**: `Unbound constructor`), `7x` e `x-plus-3` neanche.

In OCaml **tutto è un'espressione**: non ci sono comandi che cambiano uno stato, ogni costrutto calcola un valore. I tipi di base:

Tipo	Valori	Operazioni
`int`	`3`, `-1`	`+ - * /`, `mod` (resto)
`float`	`3.2`, `3.` (cioè 3.0)	`+. -. *. /.` con il punto, `**` (potenza)
`bool`	`true`, `false`	`&&`, `||`, `not`
`char`	`'a'` (apici)	`int_of_char 'a'` dà `97`, `char_of_int`
`string`	`"abcd"` (virgolette)	`^` concatena: `"hello" ^ "world"`
`unit`	solo `()`	come `void` in C

Su tutti i tipi si confronta con `= <> < > <= >=`.

> Trappole per chi viene dal C: l'uguaglianza è `=`, non `==`; il diverso è `<>`, non `!=`. `&&` e `||` sono **cortocircuitati**: in `e1 && e2`, se `e1` è `false` il risultato è già `false` ed `e2` non viene valutata.

OCaml **non ha overloading**: `+` lavora solo sugli `int`, e per i `float` serve `+.`. E **non ha conversioni implicite**: in C o Java `3 + 2.5` vale 5.5 (il 3 viene promosso a `float`), e il cast `(int) 3.2` tronca a 3. In OCaml `3 + 2.5` è un errore di tipo, e la conversione si scrive:

```
(float_of_int 3) +. 2.4;;   (* - : float = 5.4 *)
int_of_float 3.7;;          (* - : int = 3 *)
int_of_string "10";;        (* - : int = 10 *)
```

Due motivi: le conversioni implicite sono una fonte di errori difficili da trovare, e la type inference ha bisogno di sapere con esattezza il tipo di ogni espressione.

Il primo tipo strutturato è la **tupla**: una sequenza di valori, anche di tipi diversi, separati da virgole (le parentesi si possono omettere). I componenti si riconoscono dalla posizione, non da un nome.

```
let t = (10, "hello", 12.5);;
(* val t : int * string * float = (10, "hello", 12.5) *)
```

Il tipo è il **prodotto** dei tipi dei componenti: se vediamo `int` e `float` come gli insiemi $\mathbb{Z}$ e $\mathbb{R}$, `int * float` è il prodotto cartesiano $\mathbb{Z} \times \mathbb{R}$, e `(3, 4.2)` ne è un elemento. Una tupla si può **destrutturare** con `let`, scrivendo a sinistra un **pattern** con la stessa forma:

```
let p = (3.0, 5.0);;    (* val p : float * float = (3., 5.) *)
let (x, y) = p;;        (* val x : float = 3.   val y : float = 5. *)
x +. y;;                (* - : float = 8. *)
```

4.2 Funzioni

Come nel λ-calcolo, in OCaml le **funzioni sono valori**: possono essere il risultato di un'espressione, essere passate a un'altra funzione o restituite da una funzione. La λ diventa `fun`, il punto diventa `->`:

```
fun x -> x + 1;;          (* - : int -> int = <fun> *)
(fun x -> x + 1) 10;;     (* - : int = 11 *)
```

`fun x -> x + 1` è $\lambda x.x + 1$: una funzione **anonima** (senza nome), con argomento `x` e corpo `x + 1`. Il tipo `int -> int` dice che prende un intero e restituisce un intero; il valore è scritto `<fun>`, perché una funzione non si può stampare. In JavaScript la stessa funzione è `function(x) { return x+1 }` o `x => x + 1`; anche Java ha le *lambda expressions*. Sono tutte idee prese dal λ-calcolo.

La traduzione da λ-calcolo (con numeri e operazioni) a OCaml è una funzione $[\![\cdot]\!]$, definita per ricorsione sulla struttura dei termini come FV: un caso per ogni elemento della sintassi.

λ-termine	**OCaml**
$[\![n]\!]$ (un numero)	`n`
$[\![x]\!]$	`x`
$[\![T'$ op $T'']\!]$ (op è `+ - * /`)	$[\![T']\!]$ `op` $[\![T'']\!]$
$[\![\lambda x.T']\!]$	`fun x ->` $[\![T']\!]$
$[\![T'T'']\!]$	$[\![T']\!][\![T'']\!]$

Si traduce ogni λ-termine, ma non tutto OCaml viene da un λ-termine: la sintassi di OCaml è molto più ricca. Per esempio $[\![\lambda f.\lambda n.fn+1]\!] =$ `fun f -> fun n -> f n + 1`. Applicata a $\lambda x.2*x$ e a 10 dà lo stesso risultato della β-riduzione (l'applicazione associa a sinistra, e fn si calcola prima del $+$):

$$(\lambda f.\lambda n.fn+1)(\lambda x.2*x)10 \to (\lambda n.(\lambda x.2*x)n+1)10 \to (\lambda n.2*n+1)10 \to 20+1=21$$

```
(fun f -> fun n -> f n + 1) (fun x -> 2 * x) 10;;   (* - : int = 21 *)
```

È una **funzione di ordine superiore** (*higher-order*): prende una funzione (`f`) come parametro.

Con più parametri si scrive una catena di `fun`, e il tipo è una catena di frecce:

```
fun x -> fun y -> x + y;;   (* - : int -> int -> int = <fun> *)
```

`->` nei tipi **associa a destra**: `int -> int -> int` è `int -> (int -> int)`, una funzione che prende `x` e restituisce la funzione `fun y -> x + y`.

Per scrivere meno, `fun x y -> x + y` sta per `fun x -> fun y -> x + y`. È **zucchero sintattico**: un costrutto che non aggiunge niente al linguaggio, perché si può scrivere con altri costrutti, ma rende il codice più comodo, come `x += 1` per `x = x + 1` in C.

Siccome le funzioni sono valori, si legano a un nome con `let`. La seconda forma è zucchero per la prima ed è **il modo più comune di definire funzioni**:

```
let somma = fun x y -> x + y;;
let somma x y = x + y;;          (* val somma : int -> int -> int = <fun> *)
somma 3 4;;                      (* - : int = 7 *)
```

C'è anche `function x -> x + 1`, che prende **un solo** parametro: torna utile con il pattern matching (Sezione 5.2).

Non esiste il `return`: il corpo di una funzione è un'espressione, e il suo valore è il risultato. In un linguaggio imperativo il corpo è una sequenza di comandi, e `return` serve a interromperla e dire cosa restituire.

```
let media x y = (x +. y) /. 2.0;;
(* val media : float -> float -> float = <fun> *)
```

> Senza parentesi, `x +. y /. 2.0` calcola $x + \frac{y}{2}$: `/.` ha la precedenza su `+.` come nella matematica. `media 4. 6.` darebbe `7.` invece di `5.`.

Un `let` «nudo» vale per tutto quello che viene dopo (ambiente globale). `let ... in` invece dichiara una variabile **locale**, visibile solo nell'espressione dopo `in` (come una variabile dichiarata in un blocco `{ }` in C):

```
let z = 10 in z * z / 2;;   (* - : int = 50 *)
let w = 5 * z;;             (* Error: Unbound value z *)
```

Si legge come in matematica: «sia $z = 10$ in $z^2/2$». Con `let ... in` si definiscono anche **funzioni ausiliarie**: `let f x = 2 * x in f (f 10)` vale 40. Un esempio completo: le soluzioni di $ax^2+bx+c=0$ sono $\left(-b \pm \sqrt{\Delta}\right)/2a$ con $\Delta = b^2 - 4ac$.

```
let sol a b c =
  let delta = b *. b -. 4. *. a *. c in
  let sol' op = (op (-. b) (sqrt delta)) /. (2. *. a) in
  (sol' (+.), sol' (-.));;
(* val sol : float -> float -> float -> float * float = <fun> *)
sol 1. (-5.) 6.;;   (* - : float * float = (3., 2.) *)
```

- `delta` è calcolato una volta sola e usato due volte; `sqrt` è la radice quadrata, già definita.
- Le due soluzioni differiscono solo per l'operazione ($+$ o $-$), quindi la funzione ausiliaria `sol'` la prende come parametro `op`.
- `(+.)` è l'operatore `+.` in **forma prefissa** (prima degli operandi): `(+.) 3.5 2.0` vale `5.5`. Un operatore usato così va tra parentesi, altrimenti OCaml lo legge in forma infissa (in mezzo agli operandi). `(-. b)` è $-b$, e anche `(-5.)` va tra parentesi.

Il nome della funzione e gli argomenti sono separati da **spazi**, senza parentesi né virgole. Queste funzioni si chiamano **curried**: prendono gli argomenti uno alla volta, come nel λ-calcolo (Sezione 1.6). `somma 3 4` è `(somma 3) 4`. Si possono usare anche parentesi e virgole, ma è un'altra funzione:

Definizione	Tipo	Chiamata
`let somma x y = x + y`	`int -> int -> int`	`somma 3 4`, un argomento alla volta
`let somma (x, y) = x + y`	`int * int -> int`	`somma (3, 4)`: **un solo** argomento, una coppia, destrutturata in `x` e `y`

Il vantaggio della forma curried è l'**applicazione parziale**: passando solo il primo argomento si ottiene una funzione che aspetta il secondo. Nel λ-calcolo $(\lambda x.\lambda y.x + y)10 \to \lambda y.10 + y$.

```
somma 10;;                  (* - : int -> int = <fun> *)
let somma_dieci = somma 10;;
somma_dieci 5;;             (* - : int = 15 *)
```

Da un'unica definizione si ottengono infinite funzioni (`somma 1`, `somma 5`...). Non è come i parametri opzionali o con valore di default di altri linguaggi: lì la funzione chiamata è sempre la stessa, si evita solo di scrivere qualche argomento.

4.3 Condizioni e ricorsione

Nei linguaggi imperativi il flusso si controlla con `if` e con i cicli (`while`), che hanno senso perché c'è l'assegnamento: un `if` sceglie fra due assegnamenti, un `while` ne ripete uno. In OCaml le variabili sono **immutabili**, come le incognite di un'espressione algebrica: non c'è assegnamento, e quindi un ciclo non avrebbe niente da cambiare. Il flusso si controlla con l'espressione `if` e con la **ricorsione**, che ripete un calcolo richiamando la stessa funzione.

`if` è un'**espressione**, come `( ? : )` in C e JavaScript: ha **sempre** tutti e due i rami, `then` ed `else`, **dello stesso tipo**, e si può mettere a destra di un `let`.

```
let x = 23;;
let y = if x > 10 then 1 else 0;;   (* val y : int = 1 *)
```

> `let f x = if x > 0 then x else "errore"` è un errore di tipo: un ramo è `int`, l'altro `string`, e il tipo del risultato non si può decidere. Senza `else`, invece, `f` non saprebbe cosa restituire quando $x \leq 0$: il corpo è un'unica espressione, e dopo l'`if` non si può scrivere altro.

Una funzione ricorsiva si dichiara con `let rec`:

```
let fact n = if n <= 0 then 1 else n * fact (n - 1);;
(* Error: Unbound value fact  (Hint: ... add the rec keyword) *)
let rec fact n = if n <= 0 then 1 else n * fact (n - 1);;
(* val fact : int -> int = <fun> *)
```

Il motivo: `let fact n = e` sta per `let fact = fun n -> e`, e mentre si valuta `fun n -> e` il nome `fact` non esiste ancora. `rec` rende il nome visibile anche dentro la sua definizione; è il costrutto fix di PCF (Sezione 3.3). La definizione somiglia a quella matematica: una scelta fra due casi, e in uno la funzione richiama se stessa.

$$\mathrm{fact}(n) = \begin{cases} 1 & \text{se } n \leq 0 \\ n \cdot \mathrm{fact}(n-1) & \text{altrimenti} \end{cases}$$

```
let rec fibonacci n =
  if n = 0 || n = 1 then n else fibonacci (n - 1) + fibonacci (n - 2);;
(* metodo di Euclide *)
let rec mcd n m = if m = 0 then n else mcd m (n mod m);;
```

Una funzione può usare solo funzioni definite prima. Due funzioni che si chiamano a vicenda (**mutuamente ricorsive**) si definiscono insieme, unite da `and`:

```
let rec pari n = if n = 0 then true else dispari (n - 1)
and dispari n = if n = 0 then false else pari (n - 1);;
pari 6;;      (* - : bool = true *)
```

4.4 Type inference e polimorfismo

Il toplevel dà il tipo di ogni espressione senza che lo scriviamo. Lo ricava da due certezze: i **valori** hanno un tipo preciso (`5.` è `float`), e gli **operatori** lavorano su tipi precisi (`+` solo su `int`, `+.` solo su `float`). Da qui deduce il tipo delle variabili: in `x + y`, `x` e `y` devono essere `int`. Procede dalle foglie dell'albero di sintassi verso la radice, tenendo una lista «variabile ↦ tipo». Se la stessa variabile compare due volte, i due tipi devono coincidere, o essere **unificabili** (uno più generico dell'altro, si prende il meno generico); altrimenti è un errore di tipo. `(x+1)-(x+4)` va bene, `(x+1)-(x+.4.0)` no: la prima `x` è `int`, la seconda `float`.

Con le funzioni si guardano anche argomenti e risultati. In `fun f -> fun n -> f n + 1` (l'applicazione ha la precedenza su `+`, quindi il corpo è `(f n) + 1`): `f n` è un operando di `+`, quindi è `int`; `f` è applicata, quindi è una funzione; di `n` non si sa niente, e il suo tipo resta un generico `'a`. Il risultato è `('a -> int) -> 'a -> int`.

Il metodo si fa a mano con i **vincoli**: una **variabile di tipo** (un tipo ancora sconosciuto) per ogni parametro e una per il risultato, poi un'equazione per ogni pezzo della funzione, poi si risolve.

```
let sum_if_true test first second =
  (if test first then first else 0) + (if test second then second else 0);;
```

Il tipo ha la forma `TEST -> FIRST -> SECOND -> RIS`.

Vincolo	Da dove viene
`TEST = FIRST -> bool`	`test first` è la condizione del primo `if`
`TEST = SECOND -> bool`	`test second` è la condizione del secondo `if`
`FIRST = int`	i rami del primo `if` sono `first` e `0`
`SECOND = int`	i rami del secondo `if` sono `second` e `0`
`RIS = int`	il risultato di `+` è `int`

Risolvendo: `FIRST = SECOND = int`, `TEST = int -> bool`, `RIS = int`, cioè `(int -> bool) -> int -> int -> int`. La funzione somma `first` e `second`, ma solo quelli che superano il controllo `test`; gli altri contano 0. Una funzione che restituisce `true` o `false`, come `test`, si chiama **predicato**. Passando predicati diversi la stessa funzione somma solo i pari, solo i positivi, o altro:

```
sum_if_true (fun x -> x mod 2 = 0) 3 4;;     (* - : int = 4 *)
sum_if_true (fun x -> x > 0) 3 (-4);;        (* - : int = 3 *)
```

Il predicato non deve essere per forza anonimo: va bene anche una funzione con nome, anche ricorsiva, come una `potenza_di_due`.

A volte i vincoli non bastano a fissare un tipo concreto:

```
let id x = x;;    (* val id : 'a -> 'a = <fun> *)
id 4;;            (* - : int = 4 *)
id "ciao";;       (* - : string = "ciao" *)
```

`id` è **polimorfa**: si applica a argomenti di tipi diversi. `'a` (si legge «alfa») è una **variabile di tipo**: al suo posto può andare un tipo qualsiasi, ma se compare più volte va sostituita sempre con lo stesso tipo. Quindi `'a -> 'a` dice che il risultato ha lo stesso tipo dell'argomento, qualunque sia. A ogni applicazione OCaml **specializza** `'a` sul tipo dell'argomento.

```
let first_if_true test first second = if test first then first else second;;
```

Vincoli: `TEST = FIRST -> bool` (la condizione), `FIRST = SECOND` (i due rami), `RIS = FIRST` (il ramo `then`). Nessuno fissa un tipo concreto: `('a -> bool) -> 'a -> 'a -> 'a`.

> Basta un ramo concreto per cambiare tutto: con `then 2` al posto di `then first` si aggiunge `RIS = SECOND = int`, mentre `FIRST` resta libero. Il tipo diventa `('a -> bool) -> 'a -> int -> int`.

```
let maggiore x y = if x > y then x else y;;
(* val maggiore : 'a -> 'a -> 'a = <fun> *)
maggiore 6 5;;                  (* - : int = 6 *)
maggiore "albero" "gatto";;     (* - : string = "gatto" *)
maggiore (fun x -> x) (fun x -> x + 1);;
(* Exception: Invalid_argument "compare: functional value". *)
```

`>` si può usare su qualunque tipo, quindi `x` e `y` sono `'a` (lo stesso per tutti e due). Con due funzioni il controllo dei tipi passa, ma **durante l'esecuzione** il confronto solleva un'**eccezione** (una situazione anomala a tempo di esecuzione). Anche `(fun x -> x) = (fun y -> y)` fa lo stesso. Non è un limite

di OCaml: due funzioni f e g sono uguali se $f(x) = g(x)$ per ogni x, e un algoritmo che lo decidesse risolverebbe il problema della fermata. È **indecidibile**.

Un'**annotazione di tipo** `(x : int)` dice al compilatore il tipo voluto, e può restringere il polimorfismo (le parentesi sono obbligatorie):

```
let maggiore (x : int) (y : int) : int = if x > y then x else y;;
(* val maggiore : int -> int -> int = <fun> *)
```

Così `maggiore` non si può più applicare a funzioni. Per averla su `int` e su `string`, e non su altro, servono due funzioni con **nomi diversi** (`maggiore_int`, `maggiore_string`): OCaml non ha **overloading**, cioè non permette due funzioni con lo stesso nome che differiscono per numero o tipo dei parametri (in Java e C# si può).

Anche TypeScript ha la type inference con tipi polimorfi, ma fa scelte diverse:

	TypeScript	**OCaml**
Identità	`id(x: any): any` spegne ogni controllo (accetta `12 * id("hello")`) e non dice che il risultato ha il tipo dell'argomento	`'a -> 'a`
Polimorfismo	esplicito: con i *generics* si scrive `id<T>(x: T): T`	implicito: `'a` lo deduce il compilatore
Due tipi possibili	tipo unione `number | string`: accetta `maggiore(3, "hello")`, e rifiuta `maggiore(2, 4) / 2` perché non sa se il risultato è un numero	due funzioni distinte

OCaml è più rigoroso: vuole il tipo preciso (anche polimorfo) di ogni espressione, quindi niente conversioni implicite né tipi «ambigui». TypeScript lascia più libertà per restare vicino a JavaScript, e qualche errore in più passa. Nessuno dei due è «migliore»: hanno scopi diversi.

5 Liste e pattern matching

Cosa devi saper fare

- costruire liste con `[ ; ]`, `::` e `@` e dire il loro tipo
- dire se un valore fa match con un pattern, e scrivere `match` esaustivi
- scrivere funzioni ricorsive su liste, anche con l'accumulatore (ricorsione in coda)
- usare `exists`, `for_all`, `filter`, `map`, `fold_right`, `fold_left`, e scegliere fra le due fold

5.1 Liste

Una **lista** è una sequenza finita e **immutabile** di valori **dello stesso tipo**. Si scrive fra parentesi quadre, con gli elementi separati da `;`. Se gli elementi hanno tipo `T`, la lista ha tipo `T list`. Immutabile vuol dire che non si possono aggiungere, togliere o cambiare elementi: ogni operazione legge una lista e ne costruisce una nuova.

```
[3; 5; -1; 9];;                  (* - : int list = [3; 5; -1; 9] *)
["cane"; "gatto"];;              (* - : string list *)
[(1, "lun"); (2, "mar")];;       (* - : (int * string) list *)
[[1; 2]; [2; 1]];;               (* - : int list list *)
[];;                             (* - : 'a list = [] *)
```

La lista vuota `[]` ha tipo `'a list`: non ha elementi che fissino il tipo.

> `[1, 2]` non è una lista di due interi: la virgola costruisce una coppia, quindi è una lista con **un solo** elemento, `(1, 2)`, di tipo `(int * int) list`.

Il tipo di una lista si trova **unificando** i tipi degli elementi: si cerca il tipo più generale compatibile con tutti.

```
[[]; []];;          (* - : 'a list list *)
[[]; []; [3]];;     (* - : int list list *)
[(fun x -> x + 1); (fun x -> x)];;   (* - : (int -> int) list *)
```

Finché ci sono solo liste vuote il tipo resta generico; appena entra `[3]` diventa `int list list`. Allo stesso modo `fun x -> x` (tipo `'a -> 'a`) si specializza in `int -> int`. Con tre funzioni:

```
let f x y = 3;;        (* val f : 'a -> 'b -> int *)
let g x y = x + 1;;    (* val g : int -> 'a -> int *)
let h x y = y + 1;;    (* val h : 'a -> int -> int *)
[f; g];;               (* - : (int -> 'a -> int) list *)
[g; h];;               (* - : (int -> int -> int) list *)
```

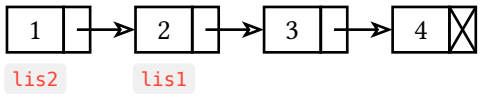
`g` fissa il primo parametro a `int`, `h` il secondo: insieme li fissano tutti e due. Il tipo trovato così si chiama **most general unifier** (MGU): il tipo più generale che rispetta tutti i vincoli.

Il costruttore fondamentale è `::` (*cons*): se `l` ha tipo `T list` ed `e` ha tipo `T`, `e :: l` è la lista che ha `e` come primo elemento, seguito dagli elementi di `l`.

```
4 :: [3; 2; 1];;           (* - : int list = [4; 3; 2; 1] *)
1 :: 2 :: 3 :: 4 :: [];;   (* - : int list = [1; 2; 3; 4] *)
```

`[1; 2; 3; 4]` è zucchero per `1 :: (2 :: (3 :: (4 :: [])))`, e le parentesi si tolgono perché `::` associa a destra. Si vede la natura **induttiva** delle liste: si parte da `[]` e si mette davanti 4, poi 3, poi 2, poi 1.

Si dice spesso che `::` «aggiunge un elemento in testa», ma non è esatto: le liste sono immutabili, e `e :: l` è una lista **nuova**. Eppure non copia niente. Le liste sono **liste concatenate**: ogni nodo contiene un elemento e un puntatore al nodo dopo; la croce è la fine della lista.

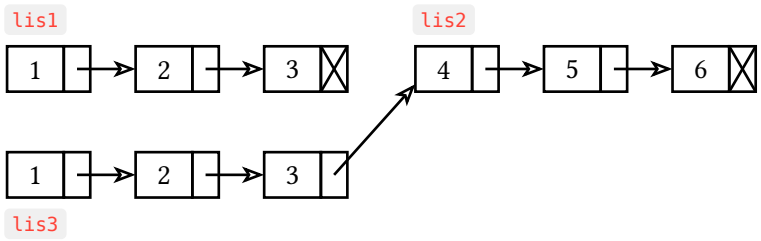


`lis1 = [2; 3; 4]` e `lis2 = 1 :: lis1`: il nuovo nodo punta alla vecchia lista

Le due liste **condividono** i nodi di `lis1`. Siccome nessuno può modificarli, per il programmatore è come se fossero due liste distinte, e non si spreca memoria.

La **concatenazione** `lis1 @ lis2` (*append*) mette in fila due liste. Questa volta la prima lista **viene copiata**, nodo per nodo:

```
[1; 2; 3] @ [4; 5; 6];;    (* - : int list = [1; 2; 3; 4; 5; 6] *)
```



`lis3 = lis1 @ lis2`: i nodi di `lis1` sono copiati, l'ultima copia punta a `lis2`

Se invece `@` cambiasse il puntatore dell'ultimo nodo di `lis1` per farlo puntare a `lis2`, `lis1` non sarebbe più `[1; 2; 3]`: la distruggerebbe.

Il modulo `List` della libreria standard ha molte funzioni pronte, che si chiamano con `List.` davanti:

Funzione	Cosa fa	Esempio
`List.length`	lunghezza	`List.length [5; 2; 1]` = `3`
`List.hd`	*head*, il primo elemento	`List.hd [5; 2; 1]` = `5`
`List.tl`	*tail*, la lista senza il primo	`List.tl [5; 2; 1]` = `[2; 1]`
`List.rev`	rovescia	`List.rev [5; 2; 1]` = `[1; 2; 5]`

5.2 Pattern matching

Per leggere gli elementi di una lista bisogna **destrutturarla**. Lo strumento è il **pattern matching**, che funziona su tutti i tipi, non solo sulle liste:

```
match EXP with
| P_1 -> EXP_1
| P_2 -> EXP_2
...
| P_N -> EXP_N
```

Ogni riga `P_i -> EXP_i` è un **ramo** (o caso). Il valore di `EXP` si confronta con i pattern **dall'alto in basso**: al primo `P_i` che **combacia** (fa *match*) si valuta `EXP_i`, e gli altri rami si ignorano. Un pattern è uno di questi:

- un valore di un tipo di base (non una funzione): `true`, `0`, `2.3`, `'a'`, `"abc"`;
- una variabile: `x`, `y`...;
- una tupla di pattern: `(P_1, ..., P_N)`;
- una lista di lunghezza fissa: `[P_1; ...; P_N]`;
- una lista con cons: `P_1 :: P_2`;
- la **wildcard** `_`, che combacia con tutto.

Match. Un valore v fa match con un pattern P se P è `_`, oppure $P = v$, oppure si possono sostituire le variabili di P con dei valori ottenendo P' con $P' = v$.

In pratica P «copre» la forma di v, e intanto lega le sue variabili ai pezzi di v corrispondenti.

Pattern	Fanno match	Non fanno match
`1`	`1`	`0`, `2`, `3`
`(3, 2)`	`(3, 2)`	`(2, 3)`, `(4, 2)`
`(x, y)`	`(3, 2)`, `(2, 3)`, `(4, 2)`	`(1, 3, 2)`, `(1, 4, 3, 2)`
`(x, 2)`	`(3, 2)`, `(4, 2)`, `("ciao", 2)`	`(2, 3)`, `(4, 2, 1)`
`[]`	`[]`	`[3]`, `[1; 5]`, `['a'; 'b'; 'c']`
`[3]` o `3 :: []`	`[3]`	`[]`, `[1; 5]`, `['a'; 'b'; 'c']`
`3 :: x`	`[3]`, `[3; 4; 5]`	`[]`, `[1; 5]`, `['a'; 'b'; 'c']`
`x :: y`	`[3]`, `[3; 4; 5]`, `[1; 5]`, `['a'; 'b'; 'c']`	`[]`
`_`	`5`, `true`, `"abc"`, `(3, "hello")`, `[3; 2; 4]`, `[]`	

In `x :: lis'` la variabile `x` è il **primo elemento** e `lis'` è **la lista degli altri**: in una lista di interi `x` è un `int` e `lis'` una `int list`. Una funzione con il `match` si legge come una funzione definita per casi:

```
let rec fibonacci n = match n with
  | 0 -> 0
  | 1 -> 1
  | _ -> fibonacci (n - 1) + fibonacci (n - 2);;   (* tutti gli altri *)
let my_or x y = match (x, y) with        (* match su una coppia *)
  | (false, false) -> false
  | _ -> true;;
let inizia_con_zero lis = match lis with
  | 0 :: _ -> true
  | _ -> false;;
let lunghezza_uno lis = match lis with
  | _ :: [] -> true
  | _ -> false;;
```

È bene (ma non obbligatorio) che i pattern siano **esaustivi**: ogni valore possibile deve combaciare con almeno un ramo. Se manca un caso il compilatore dà un *warning* (un avviso: compila lo stesso) con un esempio di valore scoperto; un warning arriva anche per un ramo che non può mai scattare, per esempio uno messo dopo `_`.

```
let giorno n = match n with
  | 1 -> "lunedì" | 2 -> "martedì" | 3 -> "mercoledì" | 4 -> "giovedì"
  | 5 -> "venerdì" | 6 -> "sabato" | 7 -> "domenica";;
(* Warning 8: this pattern-matching is not exhaustive.
   Here is an example of a case that is not matched: 0 *)
giorno 7;;   (* - : string = "domenica" *)
giorno 8;;   (* Exception: Match_failure ... *)
```

Quando una funzione fa subito il `match` sul suo argomento si usa **`function`**, che prende i pattern al posto del parametro (il nome del parametro sparisce). Con più parametri, `function` fa il match sull'ultimo:

```
let is_empty = function
  | [] -> true
  | _ -> false;;
let is_first n = function       (* val is_first : 'a -> 'a list -> bool *)
  | [] -> false
  | x :: _ -> x = n;;
```

Fin qui il `match` sembra uno `switch`, solo che controlla anche la **forma** del valore (per esempio «lista non vuota»). La vera forza è un'altra: intanto che controlla, **smonta** la struttura e dà un nome ai pezzi, cioè lega le variabili del pattern, e le si può usare nel ramo. Per le tuple basta il `let`, che fa un match implicito:

```
let primo (x, _) = x;;   (* val primo : 'a * 'b -> 'a *)
let testa lis = match lis with x :: _ -> x;;
(* Warning 8: ... not matched: []   val testa : 'a list -> 'a *)
```

`testa` è una **funzione parziale**: funziona solo sulle liste non vuote.

Più pattern con la stessa espressione si **accorpano** con `|`. Le variabili però devono comparire in **tutti** i pattern accorpati:

```
let giorno_festivo n = match n with
  | 1 | 2 | 3 | 4 | 5 -> "feriale"
  | 6 | 7 -> "festivo"
  | _ -> "errore";;
let somma_coppia c = match c with
  | (x, 0) | (0, x) -> x
  | (x, y) -> x + y;;
let lunghezza_uno_due lis = match lis with
  | x :: [] | x :: y :: [] -> true
  (* Error: Variable y must occur on both sides of this | pattern *)
  | _ -> false;;
```

Se fosse permesso, in `x :: [] | x :: y :: [] -> x + y` la `y` non avrebbe valore quando la lista ha un solo elemento. Il compilatore controlla i pattern senza guardare l'espressione dopo `->`, quindi segnala l'errore anche qui, dove il ramo restituisce solo `true`. Si risolve separando i due rami, oppure, visto che le variabili non servono, con le wildcard: `| _ :: [] | _ :: _ :: [] -> true`.

Esercizio 12
Determinare il tipo delle seguenti espressioni OCaml:

```
(* A *) (3, 10.2) :: (3, 2.1) :: [];;
(* B *) [] :: [];;
(* C *) fun x -> fun y -> (x, if y > 0 then x else y < 10);;
(* D *) (fun x -> fun y -> x + y) 10;;
(* E *) let f x y z =
          if x then []
          else match y with
          | (0, 0) -> []
          | (n1, n2) -> z :: [n1; n2];;
(* F *) let g x y z =
          let k = 10 in
          if x = k then y
          else [z];;
```

5.3 Ricorsione sulle liste

Il pattern matching serve a scrivere funzioni ricorsive sulle liste. Una lista ha due forme, e la funzione ha un ramo per ciascuna, come una dimostrazione per induzione:

- **caso base**: la lista vuota `[]`;
- **caso ricorsivo**: `x :: lis'`, dove la funzione si richiama su `lis'`, più corta.

```
let rec length lis = match lis with
  | [] -> 0
  | _ :: lis' -> 1 + length lis';;
let rec somma lis = match lis with
  | [] -> 0
  | x :: lis' -> x + somma lis';;
(* val contains : 'a -> 'a list -> bool *)
let rec contains x lis = match lis with
  | [] -> false
  | y :: lis' -> if y = x then true else contains x lis';;
```

Anche `@` si scrive così: se la prima lista è vuota il risultato è la seconda; altrimenti si mette il primo elemento davanti alla concatenazione del resto.

```
let rec append lis1 lis2 = match lis1 with
  | [] -> lis2
  | x :: lis1' -> x :: append lis1' lis2;;
```

Ogni chiamata stacca un elemento da `lis1` e lo tiene in sospeso; quando `lis1` è vuota, i risultati risalgono:

```
append [1; 2] [3; 4] = 1 :: append [2] [3; 4]
                     = 1 :: (2 :: append [] [3; 4])
                     = 1 :: (2 :: [3; 4])
                     = [1; 2; 3; 4]
```

Il tempo è **lineare** nella lunghezza di `lis1`: una chiamata per elemento. Per **rovesciare** una lista l'idea più diretta è «rovescio il resto e metto il primo in fondo»:

```
let rec rev lis = match lis with
  | [] -> []
  | x :: lis' -> rev lis' @ [x];;
```

```
rev [1; 2; 3] = rev [2; 3] @ [1]
              = (rev [3] @ [2]) @ [1]
              = ((rev [] @ [3]) @ [2]) @ [1]
              = ([3] @ [2]) @ [1] = [3; 2] @ [1] = [3; 2; 1]
```

Funziona, ma è lenta: ogni `@` ricopia tutta la parte già rovesciata. Meglio un secondo parametro, l'**accumulatore**, che parte da `[]` e costruisce il risultato passo per passo:

```
let rev lis =
  let rec rev_accum lis1 lis2 = match lis1 with
    | [] -> lis2
    | x :: lis1' -> rev_accum lis1' (x :: lis2)
  in rev_accum lis [];;
```

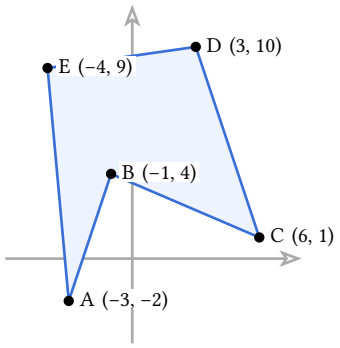
A ogni chiamata il primo elemento di `lis1` passa in testa a `lis2`, quindi gli elementi arrivano in ordine inverso:

```
rev_accum [1; 2; 3] []
→ rev_accum [2; 3] [1]
→ rev_accum [3] [2; 1]
→ rev_accum [] [3; 2; 1]  =  [3; 2; 1]
```

In `rev_accum` la chiamata ricorsiva è **l'ultima operazione**: dopo non resta niente da fare. Una funzione così si dice **ricorsiva in coda** (*tail recursive*). In `somma`, invece, dopo la chiamata resta da fare `x + ...`. Il vantaggio è a tempo di esecuzione: ogni chiamata di funzione occupa nello **stack** (la pila della memoria) un **record di attivazione** con i suoi parametri, ma se non resta lavoro in sospeso basta **un solo** record, riusato a ogni chiamata. Il programma è più veloce, usa meno memoria, e soprattutto non rischia lo **stack overflow** (lo stack pieno, che ferma il programma) su liste lunghe. L'accumulatore è il modo tipico di scrivere funzioni ricorsive efficienti sulle liste.

Un esempio più grosso: l'**area di un poligono** qualsiasi, date le coordinate dei vertici in senso antiorario. Il metodo:

1. si copia il primo vertice in fondo alla lista;
2. si somma ogni x per la y del vertice dopo: $\sum x_i y_{i+1}$;
3. si somma ogni y per la x del vertice dopo: $\sum y_i x_{i+1}$;
4. l'area è la differenza delle due somme, divisa per 2.



Vertice	x_i, y_i	$x_i y_{i+1}$	$y_i x_{i+1}$
A	−3, −2	−12	2
B	−1, 4	−1	24
C	6, 1	60	3
D	3, 10	27	−40
E	−4, 9	8	−27
	somma	**82**	**−38**

L'area è $(82 - (-38))/2 = 60$. Nel codice, invece di copiare il primo vertice in fondo (con `@`, che copierebbe tutta la lista), lo si passa come parametro `(x0, y0)` e lo si usa quando si arriva all'ultimo vertice. Le due somme si fanno in un unico passaggio, portandosi dietro una **coppia** di risultati:

```
let area pol =
  let rec calcolo (x0, y0) lis = match lis with
    | [] -> (1, 1)                        (* non succede mai *)
    | (x, y) :: [] -> (x * y0, y * x0)    (* ultimo: il dopo è il primo *)
    | (x1, y1) :: (x2, y2) :: lis' ->
        let (ris1, ris2) = calcolo (x0, y0) ((x2, y2) :: lis') in
        (ris1 + x1 * y2, ris2 + y1 * x2)
  in
  match pol with
  | [] -> 0.
  | p0 :: _ -> let (ris1, ris2) = calcolo p0 pol in
               float_of_int (ris1 - ris2) /. 2.;;
area [(-3, -2); (-1, 4); (6, 1); (3, 10); (-4, 9)];;   (* - : float = 60. *)
```

- Il ramo `[]` di `calcolo` non scatta mai (la si chiama su liste che hanno almeno un elemento), e c'è solo per rendere il match esaustivo.
- Il terzo pattern guarda **due** vertici consecutivi, `(x1, y1)` e `(x2, y2)`, e nella chiamata ricorsiva rimette in testa il secondo.
- Somme e prodotti sono su `int`; la divisione si fa su `float`, perché l'area può non essere intera.

Esercizio 13
Scrivere una funzione ricorsiva `somma_positivi` che, data una lista `lis` di interi, restituisce la somma degli elementi positivi. Ad esempio, `somma_positivi [3; 0; -1; 2; -4]` restituisce `5`.

Esercizio 14
Scrivere una funzione ricorsiva `somma_liste` che, date due liste `[x1; ...; xn]` e `[y1; ...; yn]` di uguale lunghezza, restituisca la lista `[x1+y1; ...; xn+yn]`. Se le liste hanno lunghezze diverse, la funzione restituisce la lista vuota.

Esercizio 15
Scrivere una funzione ricorsiva `parentesi_bilanciate` che, data una lista `lis` di caratteri, restituisce `true` se la lista contiene una sequenza di caratteri `'('` e `')'` bilanciata. Ad esempio, `parentesi_bilanciate ['('; '('; ')'; '('; '('; ')'; ')'; ')']` restituisce `true`, mentre `parentesi_bilanciate ['('; '('; ')'; ')'; ')']` restituisce `false`.

5.4 Funzioni di ordine superiore sulle liste

Sulle liste l'unico modo di ripetere un calcolo è la ricorsione, ma molte funzioni ricorsive hanno **lo stesso schema**. Queste tre differiscono solo per la condizione:

```
let rec contiene_zero lis = match lis with
  | [] -> false
  | x :: lis' -> if x = 0 then true else contiene_zero lis';;
let rec contiene_positivo lis = ...   (* uguale, con x > 0 *)
let rec contiene_pari lis = ...       (* uguale, con x mod 2 = 0 *)
```

Tutte scorrono la lista, controllano una condizione su ogni elemento, restituiscono `true` al primo che la soddisfa e `false` se arrivano in fondo. L'idea: la condizione diventa un **predicato** `p : 'a -> bool` passato come parametro, e lo schema si scrive una volta sola.

```
(* val exists : ('a -> bool) -> 'a list -> bool *)
let rec exists p lis = match lis with
  | [] -> false
  | x :: lis' -> if p x then true else exists p lis';;
let contiene_zero lis = exists (fun x -> x = 0) lis;;
let contiene_pari lis = exists (fun x -> x mod 2 = 0) lis;;
let contains x lis = exists (fun y -> x = y) lis;;
```

La ricorsione adesso è nascosta dentro `exists`. In `contains` la funzione `fun y -> x = y` usa `x`, che non è un suo parametro: è una sua variabile libera, che viene da fuori. Quello che `exists` riceve è allora una **chiusura** (*closure*): la funzione **insieme all'ambiente** in cui è stata creata, cioè ai valori delle sue variabili libere. La chiusura «ricorda» il valore di `x` e può confrontarlo con ogni `y`. Lo stesso si ottiene con l'applicazione parziale: con `let uguale x y = x = y`, si scrive `exists (uguale x) lis`.

Con lo stesso metodo si scrivono le altre operazioni frequenti. **`forall`** controlla che **tutti** gli elementi soddisfino il predicato:

```
(* val forall : ('a -> bool) -> 'a list -> bool *)
let rec forall p lis = match lis with
  | [] -> true
  | x :: lis' -> if p x then forall p lis' else false;;
let tutti_pari lis = forall (fun x -> x mod 2 = 0) lis;;
```

`filter` tiene solo gli elementi che soddisfano il predicato:

```
(* val filter : ('a -> bool) -> 'a list -> 'a list *)
let rec filter p lis = match lis with
  | [] -> []
  | x :: lis' -> if p x then x :: filter p lis' else filter p lis';;
filter (fun x -> x mod 2 = 0) [1; 2; 3; 4];;   (* - : int list = [2; 4] *)
```

```
filter pari [1; 2; 3; 4] = filter pari [2; 3; 4]     1 è dispari: scartato
                         = 2 :: filter pari [3; 4]   2 è pari: tenuto
                         = 2 :: filter pari [4]
                         = 2 :: 4 :: filter pari []
                         = 2 :: 4 :: []  =  [2; 4]
```

`map` applica la stessa funzione `f` a ogni elemento: `map f [x1; ...; xn] = [f x1; ...; f xn]`. Qui non serve un predicato ma una funzione qualsiasi, che può anche cambiare il tipo degli elementi:

```
(* val map : ('a -> 'b) -> 'a list -> 'b list *)
let rec map f lis = match lis with
  | [] -> []
  | x :: lis' -> f x :: map f lis';;
map (fun x -> x + 1) [1; 2; 3];;      (* - : int list = [2; 3; 4] *)
map (fun (x, y) -> x) [(3, 2); (4, 7); (9, 2)];;
(* - : int list = [3; 4; 9] *)
```

Queste funzioni guardano ogni elemento per conto suo. Spesso invece bisogna **combinare** tutti gli elementi in un unico risultato: la somma, il minimo e il massimo, la concatenazione di stringhe. In JavaScript si userebbe un ciclo con una variabile che accumula il risultato parziale:

```
function somma(a) {
  var s = 0
  for (var i in a) s += a[i]
  return s
}
```

In un linguaggio funzionale non c'è una variabile da aggiornare: il risultato parziale **viaggia lungo la ricorsione**, come valore restituito o come parametro. Nella `somma` ricorsiva di prima, ogni chiamata restituisce la somma della lista dall'elemento corrente in fondo:

```
somma [3; 2; 4] = 3 + somma [2; 4]
                = 3 + (2 + somma [4])
                = 3 + (2 + (4 + somma []))
                = 3 + (2 + (4 + 0))
```

Gli elementi si sommano **dall'ultimo al primo**, cioè da destra. Scrivendo `+` in forma prefissa è `(+) 3 ((+) 2 ((+) 4 0))`; chiamando f l'operazione e a il valore del caso base, lo schema è $fx_1(fx_2(...(fx_n a)))$. È la **`fold_right`**:

```
let rec fold_right f lis a = match lis with
  | [] -> a
  | x :: lis' -> f x (fold_right f lis' a);;
(* val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b *)
let somma lis = fold_right (+) lis 0;;
```

I tre parametri: `f`, la funzione da applicare a ogni passo; `lis`, la lista; `a`, il risultato per la lista vuota (sta **a destra** della lista). A ogni passo `f` riceve l'elemento corrente e il risultato già calcolato sul resto della lista. Altri esempi:

```
let concat lis = fold_right (^) lis "";;
concat ["abc"; "def"; "gh"];;      (* - : string = "abcdefgh" *)
let minimo_massimo lis =
  let f x (min, max) =
    if x < min then (x, max) else if x > max then (min, x) else (min, max)
  in match lis with
  | [] -> (0, 0)
  | x :: lis' -> fold_right f lis' (x, x);;
minimo_massimo [3; 2; 4];;         (* - : int * int = (2, 4) *)
```

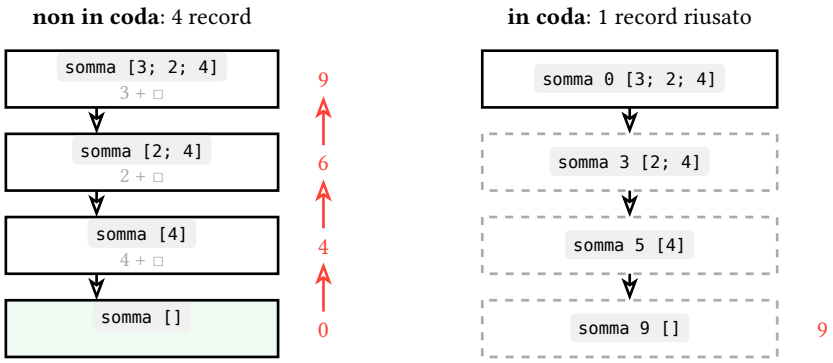
In `minimo_massimo` si porta dietro una **coppia** (minimo e massimo finora). Per il caso base non c'è un valore giusto per tutte le liste, quindi si stacca il primo elemento e si parte da `(x, x)`.

La somma si può scrivere anche al contrario, con il risultato parziale passato **come parametro** (un accumulatore, che parte da 0):

```
let rec somma a lis = match lis with
  | [] -> a
  | x :: lis' -> somma (a + x) lis';;
```

```
somma 0 [3; 2; 4] = somma (0 + 3) [2; 4]
                  = somma ((0 + 3) + 2) [4]
                  = somma (((0 + 3) + 2) + 4) []  =  9
```

Ora gli elementi si sommano **dal primo all'ultimo**, cioè da sinistra, e la funzione è ricorsiva in coda. Il disegno mostra lo stack nei due casi: a sinistra ogni chiamata lascia in sospeso un'addizione, e i risultati (in rosso) risalgono; a destra non c'è niente in sospeso, il risultato scende già fatto e basta un record riusato.

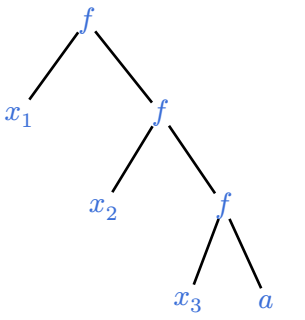


Lo stack di `somma [3; 2; 4]` nelle due versioni; in grigio l'addizione in sospeso

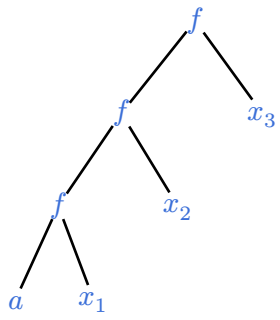
Lo schema è $f(...(f(fax_1)x_2)...)x_n$, ed è la **`fold_left`**:

```
let rec fold_left f a lis = match lis with
  | [] -> a
  | x :: lis' -> fold_left f (f a x) lis';;
(* val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a *)
let somma lis = fold_left (+) 0 lis;;
let stringa_piu_lunga lis =
  let f s x = if String.length x > String.length s then x else s in
  fold_left f "" lis;;
stringa_piu_lunga ["ciao"; "hello"; "hi"];;   (* - : string = "hello" *)
```

Qui il valore iniziale `a` sta **a sinistra** della lista, e anche in `f` l'accumulatore è il primo parametro. Regola per ricordarlo: in `fold_X` l'accumulatore sta a `X` (*left* o *right*), sia fra i parametri della fold sia fra quelli di `f`. Gli alberi delle due fold su `[x1; x2; x3]`:



`fold_right f [x1; x2; x3] a`



`fold_left f a [x1; x2; x3]`

Le due fold **non sono intercambiabili**:

	`fold_right f lis a`	`fold_left f a lis`
Ordine	dall'ultimo elemento al primo	dal primo all'ultimo
Tipo di `f`	`'a -> 'b -> 'b`: elemento, accumulatore	`'a -> 'b -> 'a`: accumulatore, elemento
In coda?	no: dopo la chiamata resta da applicare `f`	sì: un solo record nello stack
Con `(-)` su `[1; 2; 3]`, da 0	$1 - (2 - (3 - 0)) = 2$	$((0 - 1) - 2) - 3 = -6$
Con `f` che fa `x :: l`, da `[]`	`[1; 2; 3]`: la copia	`[3; 2; 1]`: la rovescia

Con `(-)` i risultati cambiano perché la sottrazione non è associativa. Nell'ultima riga `::` mette sempre in testa: `fold_right` comincia da 3 e mette davanti 2 e poi 1, quindi ricostruisce la lista nello stesso ordine; `fold_left` comincia da 1 e ci mette davanti 2 e poi 3, quindi la rovescia.

Altri due esempi. `somma_dal_fondo` costruisce la lista delle somme a partire dal fondo: se la lista accumulata non è vuota, somma l'elemento corrente al suo primo elemento e mette il risultato in testa.

```
let somma_dal_fondo lis =
  let f x l = match l with
    | [] -> [x]
    | x' :: _ -> (x + x') :: l
  in fold_right f lis [];;
somma_dal_fondo [2; 2; 2; 2; 2];;   (* - : int list = [10; 8; 6; 4; 2] *)
```

`crescente` controlla se la lista è ordinata. `f` legge un elemento alla volta, ma qui serve confrontare due elementi **consecutivi**: allora nell'accumulatore, insieme al risultato `b`, si porta dietro l'elemento precedente `prec`.

```
let crescente lis =
  let f (prec, b) x = (x, b && prec <= x) in
  match lis with
  | [] -> true
  | x :: lis' -> let (_, b) = fold_left f (x, true) lis' in b;;
crescente [1; 4; 4; 6; 9];;   (* - : bool = true *)
crescente [3; 5; 4; 7; 9];;   (* - : bool = false *)
```

Con `fold_left`, che va dal primo all'ultimo, `prec` è proprio l'elemento prima; con `fold_right` sarebbe quello dopo. La fold scorre comunque **tutta** la lista, anche quando `b` è già `false`: per fermarsi alla prima coppia fuori ordine serve una ricorsione scritta a mano.

Tutte queste funzioni sono già nel modulo `List`: `List.exists`, `List.for_all`, `List.filter`, `List.map`, `List.fold_right`, `List.fold_left`. Ce ne sono molte altre, come `List.find`, `List.sort`, `List.partition`, `List.merge`, `List.mem`. Si combinano facilmente: per sommare i numeri pari di una lista moltiplicati per 3, prima si filtrano i pari, poi si moltiplicano, poi si sommano.

```
let somma_pari_per_3 lis =
  let pari = List.filter (fun x -> x mod 2 = 0) lis in
  let per_3 = List.map (fun x -> x * 3) pari in
  List.fold_left (+) 0 per_3;;
somma_pari_per_3 [1; 2; 3; 4; 5; 6];;   (* - : int = 36 *)
```

Anche l'area del poligono diventa più corta, ma più difficile da leggere:

```
let area pol =
  let f (x1, y1) ((x2, y2), (ris1, ris2)) =
    ((x1, y1), (ris1 + x1 * y2, ris2 + x2 * y1)) in
  let (_, (ris1, ris2)) = List.fold_right f pol (List.hd pol, (0, 0)) in
  float_of_int (ris1 - ris2) /. 2.;;
```

1. `fold_right` scorre i vertici dall'ultimo al primo. L'accumulatore è una coppia di coppie: il vertice **dopo** quello corrente, `(x2, y2)`, e le due somme parziali `(ris1, ris2)`.
2. Si parte da `List.hd pol`, il primo vertice, che fa da «vertice dopo» l'ultimo (senza copiarlo in fondo), e dalle somme `(0, 0)`.
3. A ogni passo `f` aggiunge i due prodotti e restituisce come «vertice dopo» quello corrente, che servirà al passo successivo.
4. Alla fine si butta il vertice (`_`) e si usano le due somme.

Le stesse funzioni ci sono, almeno in parte, anche in altri linguaggi: in JavaScript gli array hanno `map`, `filter` e `reduce` (che è la `fold_left`, per esempio `[1, 2, 3, 4].reduce((acc, x) => acc + x)` dà 10); in Java ci sono dalla versione 8, insieme alle lambda expressions.

Esercizio 16
Risolvere l'esercizio 13 usando le funzioni `fold_left` o `fold_right`.

Esercizio 17
Risolvere l'esercizio 15 usando le funzioni higher-order su liste viste a lezione.

Esercizio 18
Utilizzando le funzioni higher-order su liste, scrivere una funzione `massimo_unico` che, data una lista `lis`, restituisca `true` se il valore massimo in `lis` è unico, e `false` se `lis` contiene più di una istanza del suo valore massimo, oppure se `lis` è vuota.

Riepilogo delle regole

Tutte le definizioni in un posto solo, per ripassare. Il nome di ogni regola è quello che si scrive negli alberi.

λ-calcolo: sintassi, variabili libere, sostituzione

$$e ::= x \mid \lambda x.e \mid ee \qquad \mathrm{FV}(x) = \{x\} \qquad \mathrm{FV}(e_1 e_2) = \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2) \qquad \mathrm{FV}(\lambda x.e) = \mathrm{FV}(e) \setminus \{x\}$$

$$x\{x := e\} = e \qquad y\{x := e\} = y \text{ se } y \neq x \qquad (e_1 e_2)\{x := e\} = (e_1\{x := e\})(e_2\{x := e\})$$

$$(\lambda x.e_1)\{x := e\} = \lambda x.e_1 \qquad (\lambda y.e_1)\{x := e\} = \lambda y.(e_1\{x := e\}) \text{ se } y \neq x,\ y \notin \mathrm{FV}(e)$$

$$(\lambda y.e_1)\{x := e\} = \lambda z.((e_1\{y := z\})\{x := e\}) \text{ se } y \neq x,\ y \in \mathrm{FV}(e),\ z \text{ fresca}$$

Riduzione standard, call-by-value, call-by-name

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\} \ \text{(App)} \qquad \frac{e \to e'}{\lambda x.e \to \lambda x.e'} \ \text{(App-Body)}$$

$$\frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \ \text{(App-1)} \qquad \frac{e_2 \to e'}{e_1 e_2 \to e_1 e'} \ \text{(App-2)}$$

$$(\lambda x.e_1)v_2 \to e_1\{x := v_2\} \ \text{(CV-App)} \qquad \frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \ \text{(CV-App-1)} \qquad \frac{e_2 \to e'}{v_1 e_2 \to v_1 e'} \ \text{(CV-App-2)}$$

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\} \ \text{(CN-App)} \qquad \frac{e_1 \to e'}{e_1 e_2 \to e' e_2} \ \text{(CN-App-1)}$$

Codifiche

$$\mathrm{TRUE} = \lambda t.\lambda f.t \qquad \mathrm{FALSE} = \lambda t.\lambda f.f \qquad \mathrm{NOT} = \lambda b.b\,\mathrm{FALSE}\,\mathrm{TRUE}$$

$$\mathrm{AND} = \lambda b.\lambda c.bc\,\mathrm{FALSE} \qquad \mathrm{OR} = \lambda b.\lambda c.b\,\mathrm{TRUE}\,c$$

$$\mathrm{IF} = \lambda c.\lambda\,\text{then}.\lambda\,\text{else}.c\ \text{then else} \qquad C_n = \lambda s.\lambda z.s^n z \qquad \mathrm{SUCC} = \lambda n.\lambda s.\lambda z.s(nsz)$$

$$\mathrm{PLUS} = \lambda m.\lambda n.\lambda s.\lambda z.ms(nsz) \qquad \mathrm{TIMES} = \lambda m.\lambda n.m(\mathrm{PLUS}\,n)C_0 \qquad Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

Linguaggio di espressioni

$$E ::= \mathrm{true} \mid \mathrm{false} \mid n \mid \mathrm{if}\,E\,\mathrm{then}\,E\,\mathrm{else}\,E \mid \mathrm{succ}\,E \mid \mathrm{pred}\,E \mid \mathrm{isZero}\,E \qquad V ::= \mathrm{true} \mid \mathrm{false} \mid n \qquad T ::= \mathrm{Bool} \mid \mathrm{Nat}$$

$$\mathrm{if\,true\,then}\,E_1\,\mathrm{else}\,E_2 \to E_1 \ \text{(If-True)} \qquad \mathrm{if\,false\,then}\,E_1\,\mathrm{else}\,E_2 \to E_2 \ \text{(If-False)}$$

$$\frac{E \to E'}{\mathrm{if}\,E\,\mathrm{then}\,E_1\,\mathrm{else}\,E_2 \to \mathrm{if}\,E'\,\mathrm{then}\,E_1\,\mathrm{else}\,E_2} \ \text{(If-Cond)}$$

$$\frac{E \to E'}{\mathrm{succ}\,E \to \mathrm{succ}\,E'} \ \text{(Succ)} \qquad \frac{m = n + 1}{\mathrm{succ}\,n \to m} \ \text{(SuccNat)} \qquad \mathrm{pred}\,0 \to 0 \ \text{(PredZero)}$$

$$\frac{E \to E'}{\mathrm{pred}\,E \to \mathrm{pred}\,E'} \ \text{(Pred)} \qquad \frac{n > 0 \quad m = n - 1}{\mathrm{pred}\,n \to m} \ \text{(PredNat)}$$

$$\frac{E \to E'}{\mathrm{isZero}\,E \to \mathrm{isZero}\,E'} \ \text{(IsZero)} \qquad \mathrm{isZero}\,0 \to \mathrm{true} \ \text{(IsZeroZero)} \qquad \frac{n > 0}{\mathrm{isZero}\,n \to \mathrm{false}} \ \text{(IsZeroSucc)}$$

$$\mathrm{true} : \mathrm{Bool} \ \text{(T-True)} \qquad \mathrm{false} : \mathrm{Bool} \ \text{(T-False)} \qquad n : \mathrm{Nat} \ \text{(T-Nat)} \qquad \frac{E : \mathrm{Bool} \quad E_1 : T \quad E_2 : T}{\mathrm{if}\,E\,\mathrm{then}\,E_1\,\mathrm{else}\,E_2 : T} \ \text{(T-If)}$$

$$\frac{E : \mathrm{Nat}}{\mathrm{succ}\,E : \mathrm{Nat}} \ \text{(T-Succ)} \qquad \frac{E : \mathrm{Nat}}{\mathrm{pred}\,E : \mathrm{Nat}} \ \text{(T-Pred)} \qquad \frac{E : \mathrm{Nat}}{\mathrm{isZero}\,E : \mathrm{Bool}} \ \text{(T-IsZero)}$$

λ-calcolo tipato (FUN) ed estensioni

$$e ::= x \mid \text{fun}\, x : \tau = e \mid \text{Apply}(e, e) \mid \text{true} \mid \text{false} \mid \text{if}\, e\, \text{then}\, e\, \text{else}\, e \mid n \mid e \oplus e \mid \text{let}\, x = e\, \text{in}\, e \mid \text{fix}\, e$$

$$v ::= \text{fun}\, x : \tau = e \mid \text{true} \mid \text{false} \mid n \qquad \tau ::= \text{Bool} \mid \text{Nat} \mid \tau \to \tau$$

$$\text{Apply}(\text{fun}\, x : \tau = e_1, v_2) \to e_1\{x := v_2\} \quad \text{(App)}$$

$$\frac{e_1 \to e_{1'}}{\text{Apply}(e_1, e_2) \to \text{Apply}(e_{1'}, e_2)} \quad \text{(App-1)} \qquad \frac{e_2 \to e_{2'}}{\text{Apply}(v_1, e_2) \to \text{Apply}(v_1, e_{2'})} \quad \text{(App-2)}$$

If-True, If-False, If-Cond come nel linguaggio di espressioni

$$\frac{e_1 \to e_{1'}}{e_1 \oplus e_2 \to e_{1'} \oplus e_2} \quad \text{(Op-1)} \qquad \frac{e_2 \to e_{2'}}{n_1 \oplus e_2 \to n_1 \oplus e_{2'}} \quad \text{(Op-2)} \qquad \frac{\delta_{\oplus}(n_1, n_2) = n}{n_1 \oplus n_2 \to n} \quad \text{(Op-3)}$$

$$\frac{e_1 \to e_{1'}}{\text{let}\, x = e_1\, \text{in}\, e_2 \to \text{let}\, x = e_{1'}\, \text{in}\, e_2} \quad \text{(Let-1)} \qquad \text{let}\, x = v\, \text{in}\, e_2 \to e_2\{x := v\} \quad \text{(Let-2)}$$

$$\frac{e \to e'}{\text{fix}\, e \to \text{fix}\, e'} \quad \text{(Fix)} \qquad \text{fix}(\text{fun}\, f : \tau = e) \to e\{f := \text{fix}(\text{fun}\, f : \tau = e)\} \quad \text{(FixFun)}$$

$$\Gamma \vdash \text{true} : \text{Bool} \quad \text{(T-True)} \qquad \Gamma \vdash \text{false} : \text{Bool} \quad \text{(T-False)} \qquad \Gamma \vdash n : \text{Nat} \quad \text{(T-Nat)} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \quad \text{(T-Var)}$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun}\, x : \tau_1 = e : \tau_1 \to \tau_2} \quad \text{(T-Fun)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2} \quad \text{(T-App)}$$

$$\frac{\Gamma \vdash e : \text{Bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if}\, e\, \text{then}\, e_1\, \text{else}\, e_2 : \tau} \quad \text{(T-If)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \oplus : \tau_1 \times \tau_2 \to \tau}{\Gamma \vdash e_1 \oplus e_2 : \tau} \quad \text{(T-Op)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let}\, x = e_1\, \text{in}\, e_2 : \tau_2} \quad \text{(T-Let)} \qquad \frac{\Gamma \vdash e : \tau \to \tau}{\Gamma \vdash \text{fix}\, e : \tau} \quad \text{(T-Fix)}$$

Proprietà

Church-Rosser: se $e \Rightarrow e_1$ e $e \Rightarrow e_2$ esiste e' con $e_1 \Rightarrow e'$ e $e_2 \Rightarrow e'$

Progresso: se $\varnothing \vdash e : \tau$ allora e è un valore oppure $e \to e'$

Conservazione: se $\Gamma \vdash e : \tau$ e $e \to e'$ allora $\Gamma \vdash e' : \tau$

Lemma di sostituzione: se $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ e $\Gamma \vdash e_2 : \tau_1$ allora $\Gamma \vdash e_3\{x := e_2\} : \tau_2$

Costrutto OCaml	Cosa fa	In C / JavaScript
`let x = e;;`	lega il nome `x` al valore di `e`, per sempre	`const x = e;`
`let x = e in e'`	`x` vale solo dentro `e'`	variabile in un blocco `{ }`
`fun x y -> e`	funzione anonima, curried	`(x, y) => e`
`let f x y = e`	funzione con nome	`function f(x, y) { return e; }`
`let rec f x = e`	funzione ricorsiva (`and` per le mutue)	ogni funzione C può essere ricorsiva
`function | P -> e ...`	funzione che fa subito il match sull'ultimo parametro	
`if c then e1 else e2`	espressione, rami dello stesso tipo, `else` obbligatorio	`c ? e1 : e2`
`match e with | P -> e1 | _ -> e2`	primo pattern che combacia; `_` combacia con tutto	`switch`, ma sulla forma
`(x, y)`, `let (x, y) = p`	tupla e sua destrutturazione	`struct` senza nomi
`[]`, `x :: l`, `l1 @ l2`	lista vuota, cons, concatenazione (copia `l1`)	
`(+.)`, `(x : int)`, `'a`	operatore prefisso, annotazione di tipo, variabile di tipo	`(int) x` è un cast, non un'annotazione

Funzione (in `List`)	Tipo	Risultato
`exists p l`	`('a -> bool) -> 'a list -> bool`	qualche elemento soddisfa `p`
`for_all p l`	`('a -> bool) -> 'a list -> bool`	tutti gli elementi soddisfano `p`
`filter p l`	`('a -> bool) -> 'a list -> 'a list`	gli elementi che soddisfano `p`
`map f l`	`('a -> 'b) -> 'a list -> 'b list`	`[f x1; ...; f xn]`
`fold_right f l a`	`('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`	`f x1 (f x2 (... (f xn a)))`
`fold_left f a l`	`('a -> 'b -> 'a) -> 'a -> 'b list -> 'a`	`f (... (f (f a x1) x2) ...) xn`, in coda